Zero Trust Access Control with Keycloak and Pomerium

PROJECT DESCRIPTION

System Overview:

Zero Trust Access Control with Keycloak and Pomerium is a security-focused infrastructure project that implements a Zero Trust Architecture (ZTA) for controlling access to internal web applications. The system replaces traditional perimeter-based security with identity-aware, context-driven access policies, ensuring every request is authenticated, authorized, and encrypted — regardless of network location.

The platform integrates Keycloak as the centralized Identity Provider (IdP) for user authentication and group management, and Pomerium as the identity-aware reverse proxy that enforces group-based access control policies. Seven React micro-frontend applications simulate real enterprise tools, all containerized using Docker and served behind Pomerium on a private Docker network.

Problem Domain:

Traditional network security models rely on perimeter defenses such as firewalls and VPNs to protect internal resources. Once a user gains access to the internal network, they often have unrestricted lateral movement — creating significant security vulnerabilities. This "castle-and-moat" approach fails in modern environments with cloud services, remote workers, and distributed microservices.

Zero Trust Architecture addresses this by requiring continuous verification of every access request, applying the principle of "never trust, always verify." This project demonstrates how open-source tools (Keycloak and Pomerium) can be combined to implement a practical Zero Trust access control system suitable for enterprise environments.

Core Technology Used:

Identity Provider: Keycloak (v21.1.1) for SSO, OIDC authentication, and group-based access management

Access Proxy: Pomerium as identity-aware reverse proxy for policy enforcement and route authorization

Containerization: Docker and Docker Compose for service orchestration and deployment

Protected Services: Seven React 18 + Vite micro-frontend applications representing enterprise resources

Authentication Protocol: OpenID Connect (OIDC) for secure identity federation

Key Capabilities:

Single Sign-On (SSO): Users authenticate once through Keycloak and access all authorized services

Role-Based Access Control (RBAC): Access policies defined per route based on user roles and groups

Identity-Aware Proxy: Pomerium validates identity on every request before forwarding to upstream services

Continuous Verification: Session-based re-authentication and token validation on every access attempt

Container-Native Deployment: Full stack runs as Docker containers for portability and scalability

Target Users:

Security Engineers: Design and implement Zero Trust access policies for organizational resources

DevOps Teams: Deploy and manage containerized security infrastructure

System Administrators: Configure identity providers and manage user access across services

APPLICATION SCENARIOS / USE CASES

1. Secure Remote Access to Internal Applications

Remote employees need to access internal dashboards, wikis, and admin panels without a traditional VPN. Pomerium acts as an identity-aware gateway, verifying user identity via Keycloak before granting access. Each route enforces role-based policies, ensuring that only authorized users can reach specific applications.

2. Role-Based Service Segmentation

An organization deploys multiple internal services (HR portal, engineering dashboard, admin panel). Using Keycloak realm roles and Pomerium policies, each service is accessible only to the appropriate user group. Engineering staff cannot access HR systems, and vice versa — enforcing the principle of least privilege.

3. Continuous Compliance Monitoring

Every access request passes through Pomerium, which logs the user identity, requested resource, access decision, and timestamp. These audit logs provide a complete trail for compliance teams to verify that security policies are enforced and detect unauthorized access attempts.

TECHNICAL ARCHITECTURE OVERVIEW

High-Level Architecture

The Zero Trust Access Control system follows a layered architecture designed for identity-aware access enforcement. All user traffic passes through a single entry point (Pomerium), which authenticates users via an external Identity Provider (Keycloak) before forwarding requests to protected applications on an isolated Docker network.

Proxy Layer (Pomerium): Pomerium serves as the identity-aware reverse proxy on port 443. It intercepts all incoming HTTPS requests, validates user sessions, and evaluates group-based access policies defined in pomerium.yaml before forwarding traffic to upstream services.

Identity Layer (Keycloak): Keycloak (v21.1.1) runs on port 8080 as the centralized Identity Provider. It manages user authentication, group membership (admin, engineer, intern), and issues OIDC tokens containing group claims via a custom Protocol Mapper.

Application Layer (React Apps): Seven React 18 + Vite micro-frontend applications serve as protected enterprise tools. Each app runs inside a Docker container on port 3000 and is only accessible through the Pomerium proxy. No authentication logic exists inside the applications.

Network Layer (Docker): All containers run on a shared private Docker network (ztca-net). Application containers use "expose" (not "ports"), making them invisible to the host network. Only Pomerium (443) and Keycloak (8080) have externally mapped ports.

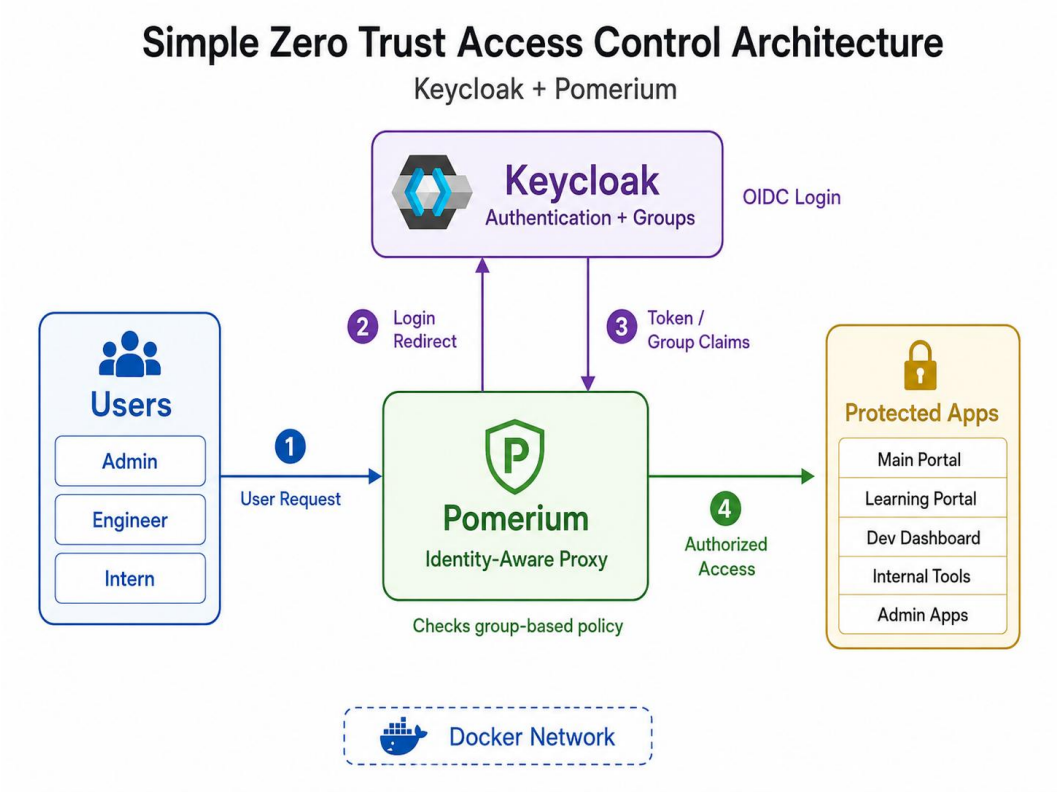


Fig 0 High-Level System Architecture — Zero Trust Access Control

Data / Request Flow

1. User Request: User navigates to https://localhost/<app-path>/ in their browser.

2. Pomerium Intercept: Pomerium receives the HTTPS request on port 443 and checks for a valid session cookie.

3. Keycloak Authentication: If no session exists, Pomerium redirects to Keycloak (port 8080) for the ztca realm login page.

4. OIDC Token Issuance: User enters credentials. Keycloak validates and issues an OIDC token containing the user's group membership claim (e.g., "groups": ["engineer"]).

5. Policy Evaluation: Pomerium extracts the group claims from the JWT and evaluates them against the route policy in pomerium.yaml (e.g., claim/groups: engineer).

6. Access Decision: If the user's group matches the policy, the request is forwarded to the upstream application container. If not, Pomerium returns 403 Forbidden.

7. Application Response: The React application serves the response back through Pomerium to the user's browser. The app itself has no knowledge of authentication.

PREREQUISITES

Software Requirements:

Docker Engine 24.x or later

Docker Compose v2.x or later

Git for version control

Web browser (Chrome, Firefox, or Edge) for testing

Text editor or IDE (VS Code recommended)

Hardware Requirements:

Minimum 8 GB RAM (recommended 16 GB)

Minimum 20 GB free disk space

Multi-core processor (2+ cores)

PRIOR KNOWLEDGE REQUIRED

Networking & Security Concepts:

TCP/IP networking, DNS resolution, and HTTP/HTTPS protocols

TLS/SSL certificates and public key infrastructure (PKI)

Zero Trust Architecture principles and security frameworks

Identity & Access Management:

OpenID Connect (OIDC) and OAuth 2.0 authorization framework

Role-Based Access Control (RBAC) design patterns

JSON Web Tokens (JWT) structure and validation

DevOps & Infrastructure:

Docker container fundamentals and Dockerfile syntax

Docker Compose for multi-container orchestration

YAML configuration file syntax

PROJECT OBJECTIVES

Deploy Keycloak as a centralized Identity Provider with OIDC support

Configure Pomerium as an identity-aware reverse proxy with policy-based routing

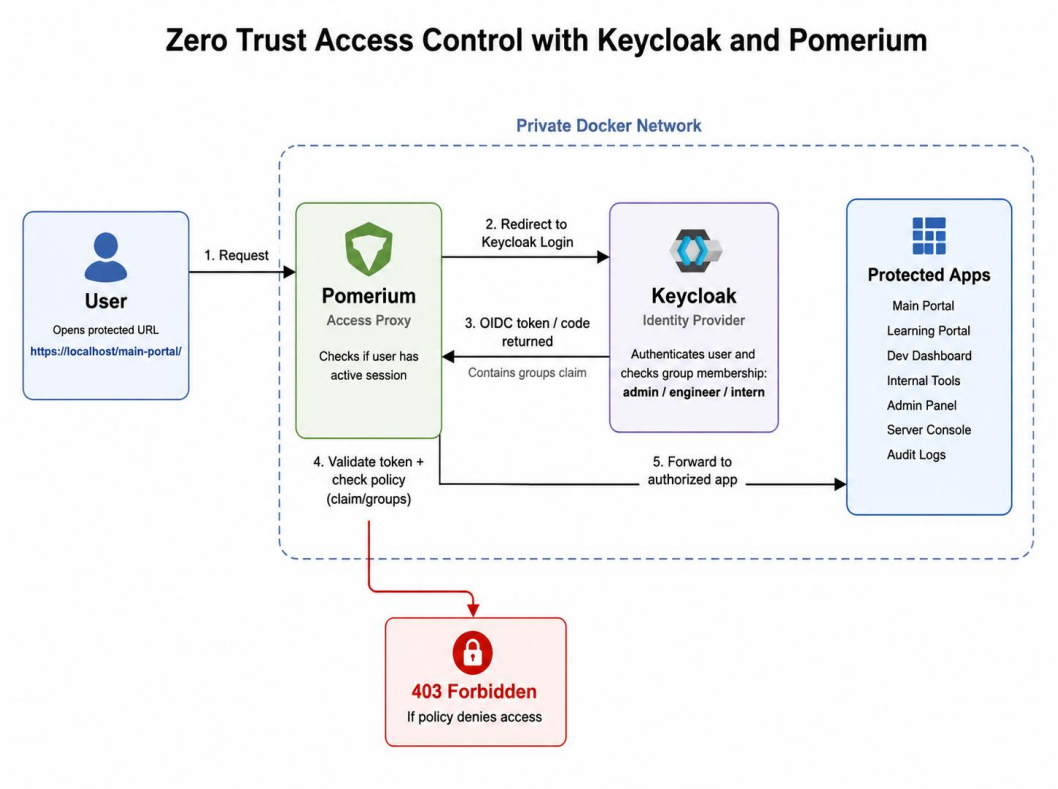
Implement role-based access control across multiple protected services

Containerize the entire stack using Docker Compose for reproducible deployment

Validate the system through comprehensive end-to-end testing

SYSTEM WORKFLOW

The following diagram illustrates the overall sequence flow of the Zero Trust Access Control system, from user request through authentication, policy evaluation, and service access:



User → Pomerium (Proxy) → Keycloak (Auth) → Pomerium (Policy) → Upstream Service

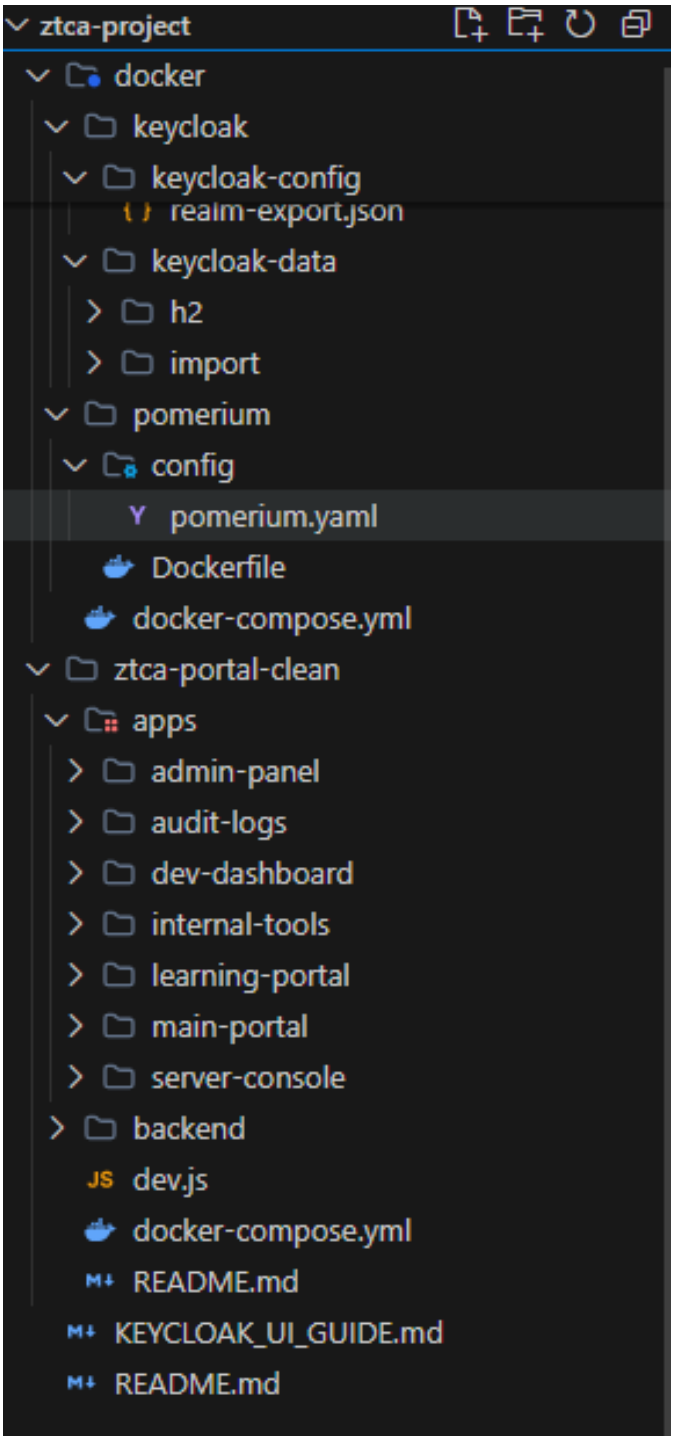
Fig 1 Zero Trust Access Control – Overall Sequence Flow

Workflow Steps:

1. User navigates to a protected service URL in their web browser
2. Pomerium intercepts the request and checks for a valid session cookie
3. If no session exists, Pomerium redirects the user to Keycloak login page
4. User authenticates with credentials; Keycloak issues an OIDC token
5. Pomerium validates the token, creates a session, and evaluates access policies
6. Authorized requests are forwarded to the upstream service; denied requests get 403 Forbidden

PROJECT STRUCTURE

```
│   ├── docker-compose.yml
│   ├── keycloak/
│   │   └── keycloak-
config/
│   │       └── realm-
export.json
│   └── pomerium/
│       └── config/
│           └──
pomerium.yaml
├── ztca-portal-clean/
│   ├── docker-compose.yml
│   ├── backend/
│   └── apps/
│       ├── main-portal/
│       ├── admin-panel/
│       ├── server-console/
│       ├── audit-logs/
│       ├── dev-dashboard/
│       ├── internal-tools/
│       └── learning-
portal/
├── KEYCLOAK_UI_GUIDE.md
└── README.md
```



Milestone 1 – Requirement Analysis & Problem Definition

1.1 Problem Definition

The core problem this project addresses is the inadequacy of traditional perimeter-based security models in modern IT environments. Organizations continue to rely on VPNs and firewalls as their primary access control mechanisms, creating a false sense of security. Once a user or device gains network access, there are often insufficient controls to prevent lateral movement across internal systems.

Key challenges in traditional access control include:

Implicit trust for internal network traffic, enabling lateral movement after initial breach

VPN-based access grants broad network access rather than application-specific permissions

Lack of continuous identity verification after initial authentication

Limited audit trail for access decisions, making compliance reporting difficult

1.2 Proposed Solution

This project implements a Zero Trust Access Control system using two open-source tools: Keycloak as the Identity Provider (IdP) and Pomerium as the identity-aware access proxy. The solution enforces the Zero Trust principle of "never trust, always verify" by requiring authentication and policy evaluation for every access request.

1.3 Functional Requirements

User Registration & Management: Keycloak shall support user creation, role assignment, and group management

SSO Authentication: Users authenticate once via Keycloak and access all authorized services

OIDC Integration: Pomerium authenticates users via Keycloak's OpenID Connect endpoint

Route-Based Access Policies: Each protected service has a configurable access policy

Access Denial Handling: Unauthorized attempts are blocked with HTTP 403 and logged

Audit Logging: All authentication events and access decisions are logged

1.4 Non-Functional Requirements

Performance: Authentication round-trip completes within 500ms under normal load

Security: All communications use TLS encryption; no plaintext credentials stored

Portability: Entire stack is deployable on any Docker-compatible host

Maintainability: Configuration is declarative (YAML) and version-controllable

1.5 Technology Stack

Component	Technology	Purpose
Identity Provider	Keycloak 21.1.1	User authentication, SSO, OIDC token issuance
Access Proxy	Pomerium (latest)	Identity-aware reverse proxy, policy enforcement
Frontend Apps	React 18 + Vite	Seven micro-frontend enterprise applications
Backend	Node.js + Express	Backend API service
Containerization	Docker + Compose	Service orchestration and deployment
Web Server	Nginx (Alpine)	Serves built React apps inside containers
Auth Protocol	OpenID Connect	Federated identity authentication

Milestone 2 – Keycloak Identity Provider

This milestone covers the setup and configuration of Keycloak as the centralized Identity Provider for the Zero Trust Access Control system. Keycloak handles user authentication, issues OIDC tokens containing group claims, and manages user identities. The Keycloak Admin Console is accessible at http://localhost:8080/admin with the master credentials (admin / admin).

2.1 Realm Setup

A Keycloak realm represents an isolated authentication domain. The "ztca" realm is created specifically for this project, separating it from the default "master" realm used for Keycloak administration. The realm is automatically imported from realm-export.json on first boot, but can also be created manually through the Admin Console.

Creating the Realm:

1. Access the Keycloak Admin Console at http://localhost:8080/admin
2. Log in with master credentials: admin / admin
3. Click the realm dropdown (top-left corner) that says "Master"
4. Click "Create Realm"
5. Enter "ztca" as the realm name and click "Create"
6. Verify "ztca" is now selected in the top-left dropdown

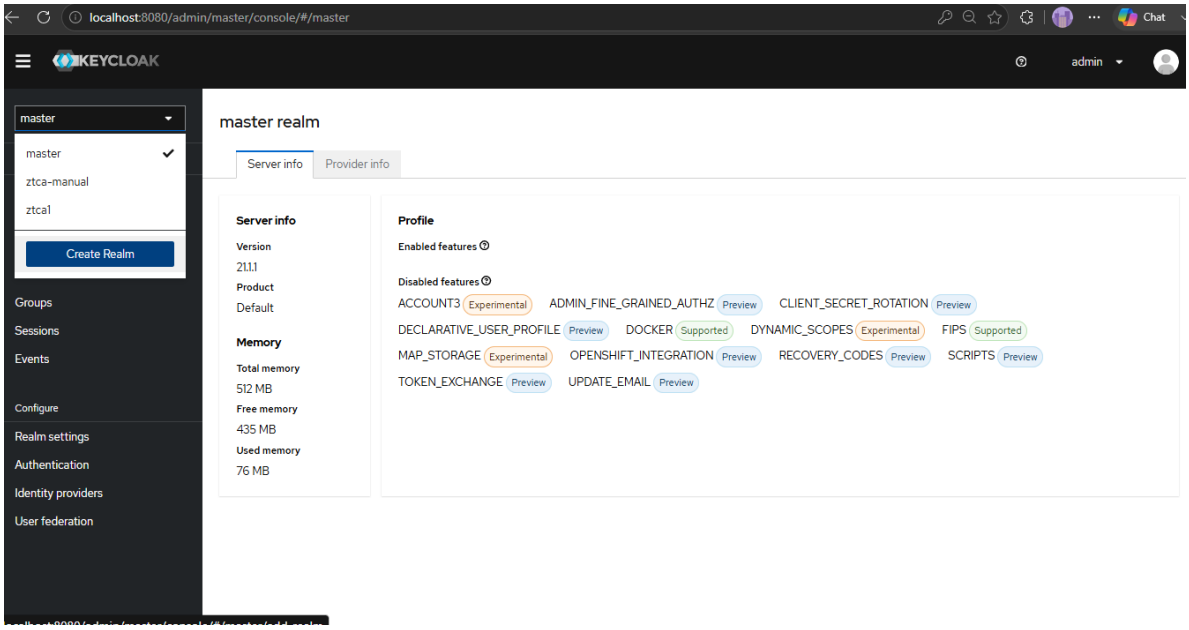


Fig 2.1 Keycloak Admin Console – Creating the "ztca" realm.

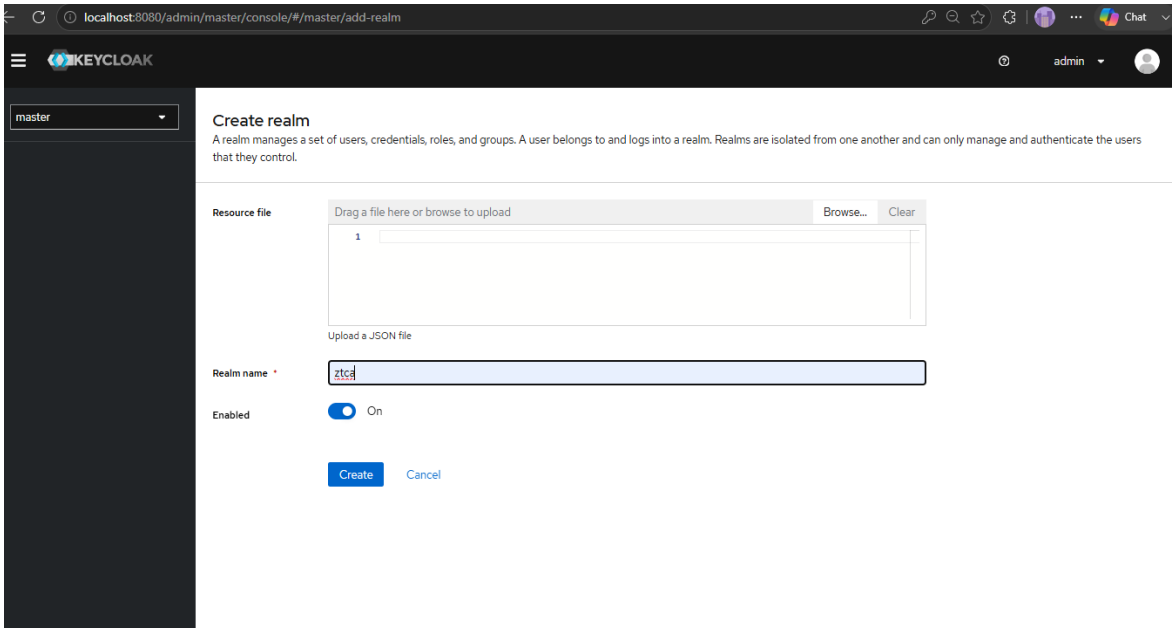


Fig 2.2 Keycloak Admin Console – Creating the "ztca" realm.

2.2 Creating Groups

Three groups are created in Keycloak to map directly to the Pomerium Zero Trust access policies. Each group determines what applications a user can access. Group membership is included in the OIDC token claims (via a Protocol Mapper) so Pomerium can evaluate group-based policies.

Group Creation Steps:

1. On the left sidebar, click "Groups"
2. Click "Create group"
3. Enter "admin" and click "Create"
4. Repeat to create the "engineer" and "intern" groups

Group Name	Access Level
admin	Full access to all 7 applications
engineer	Access to Main Portal, Dev Dashboard, Internal Tools, Learning Portal
intern	Access to Main Portal and Learning Portal only

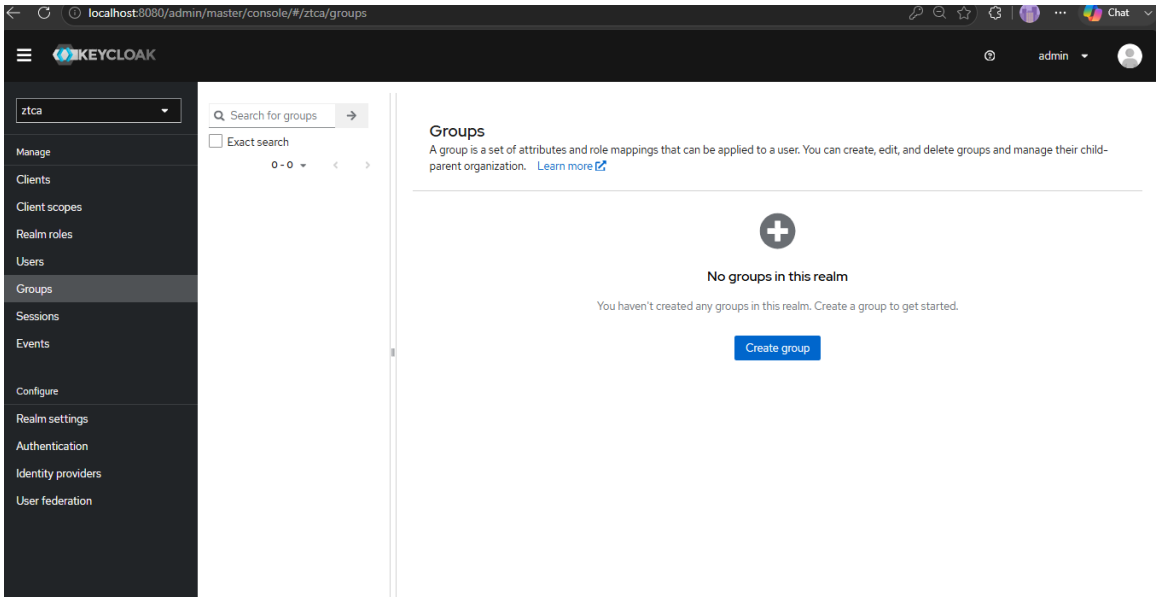


Fig 3.1 creating Groups in the Groups page

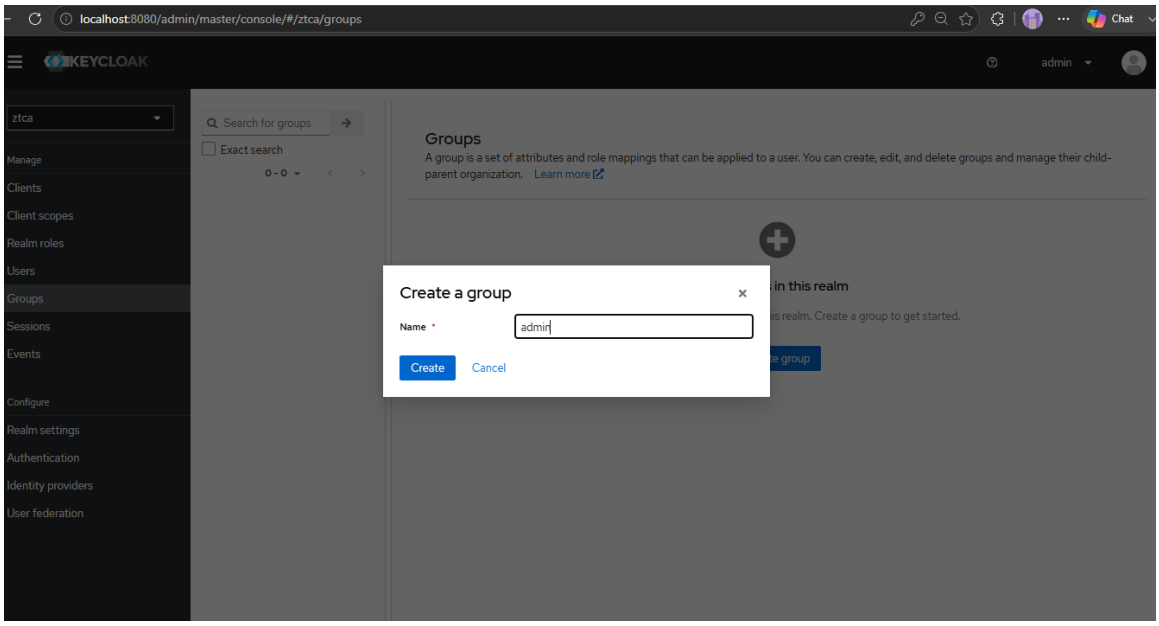


Fig 3.2 creating Groups in the Groups page

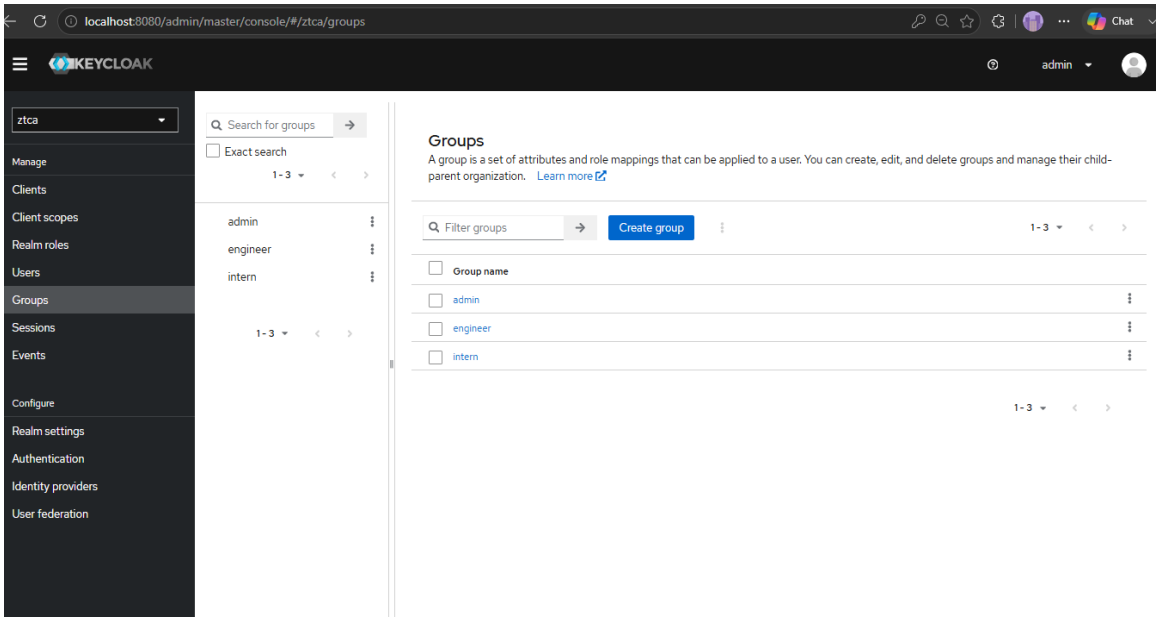


Fig 3.3 All the Groups created

2.3 Creating Users

Three test users are created, each assigned to a different group to demonstrate role-based access control through the Zero Trust system.

User Creation Process:

1. On the left sidebar, click "Users" → "Add user"
2. Enter the username and toggle "Email verified" to ON
3. Click "Create"
4. Go to the "Credentials" tab → "Set password"
5. Enter the password and toggle "Temporary" to OFF (no forced reset)
6. Click "Save" → "Save password"
7. Go to the "Groups" tab → "Join Group"
8. Select the appropriate group and click "Join"

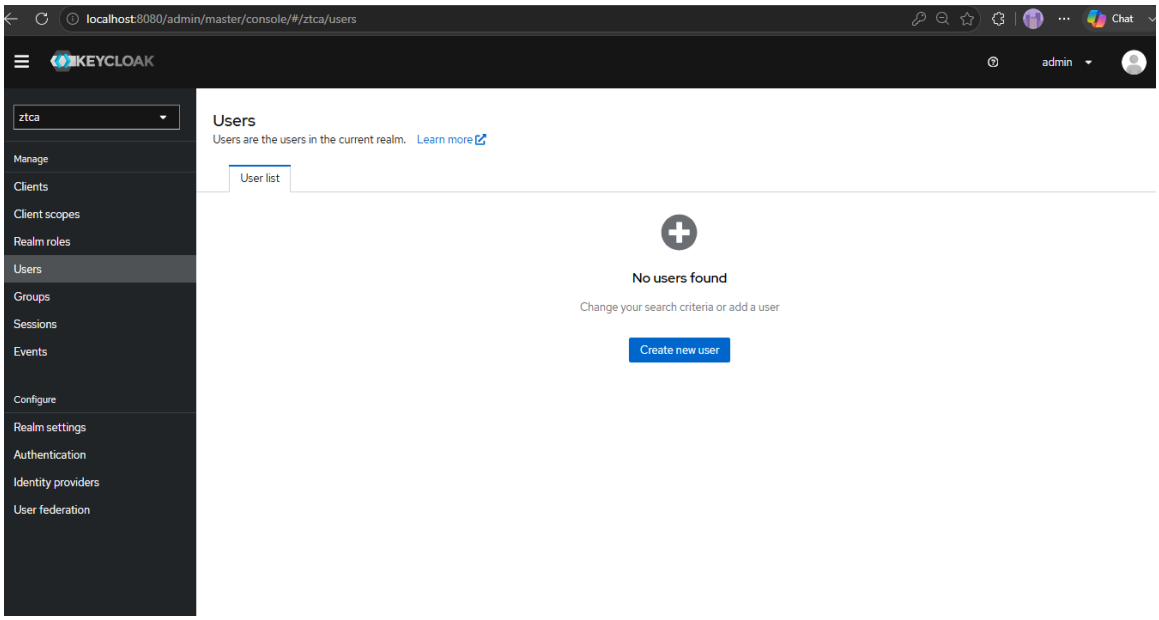


Fig 4.1 Creating users

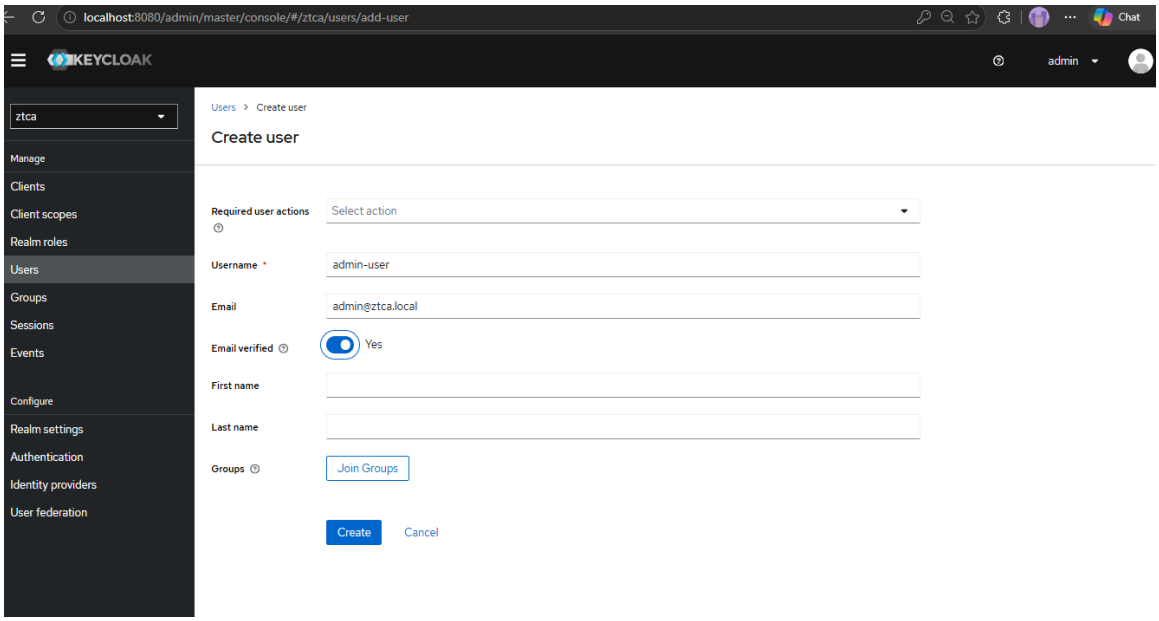


Fig 4.2 Creating users

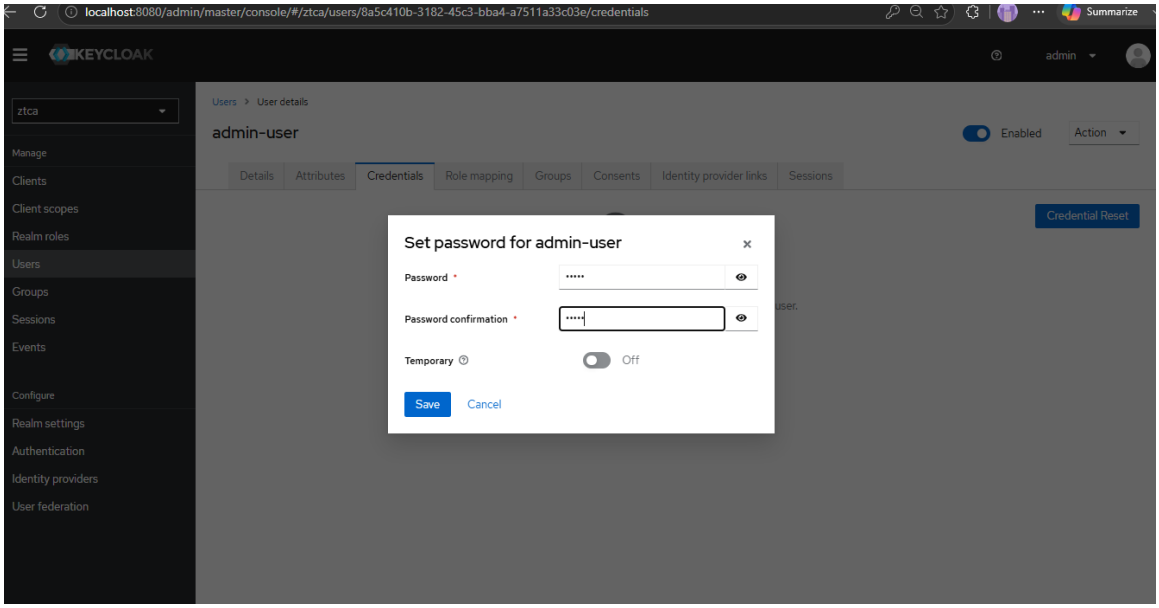


Fig 4.3 Setting the password.

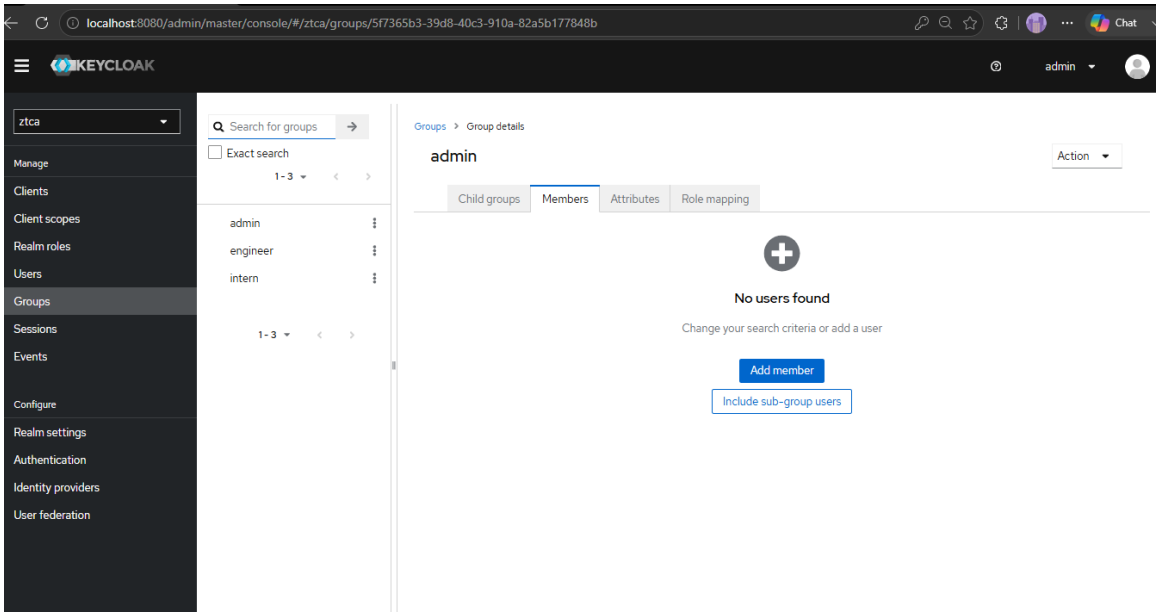


Fig 4.4. Navigating to Members tab in the Groups Page

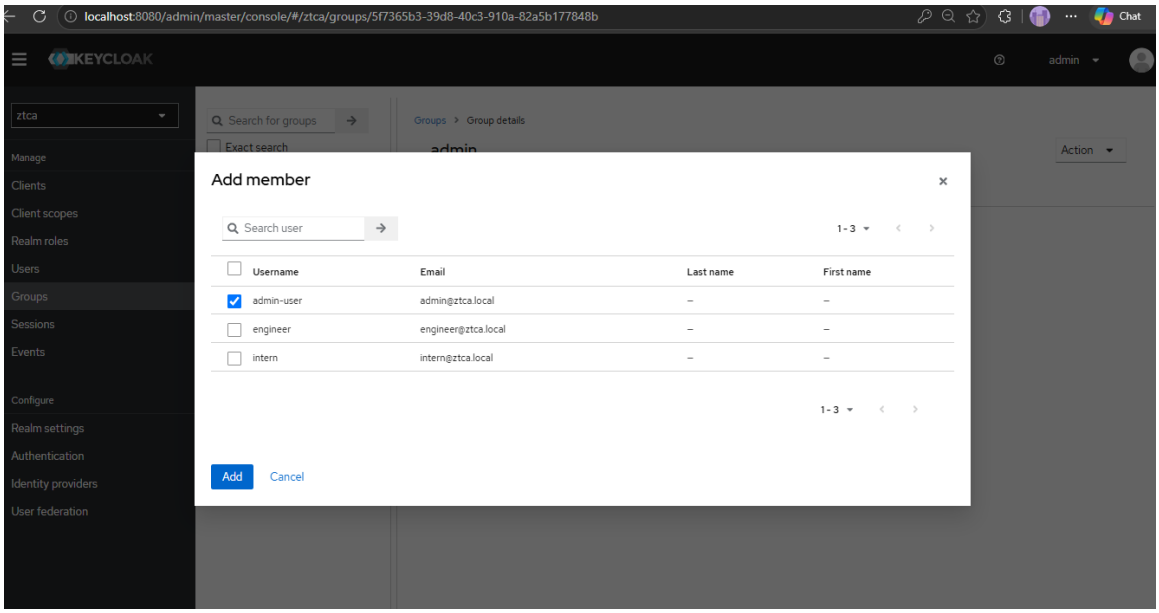


Fig 4.4 Adding Users into the respective Groups

Test Accounts:

Username	Password	Group
admin	admin	admin
engineer	engineer	engineer
intern	intern	intern

2.4 Creating the Client (ztca-client)

Pomerium needs a registered OIDC client in Keycloak to negotiate user authentication. The client is created as a "confidential" client, which generates a Client Secret that Pomerium uses to securely communicate with Keycloak.

Client Creation Steps:

1. On the left sidebar, click "Clients" → "Create client"
2. Set Client ID to "ztca-client" and click "Next"
3. Toggle "Client authentication" to ON (makes it a Confidential client)
4. Ensure "Standard flow" and "Direct access grants" are checked
5. Click "Next"
6. Set Valid Redirect URIs to * (wildcard) and press Enter
7. Click "Save"

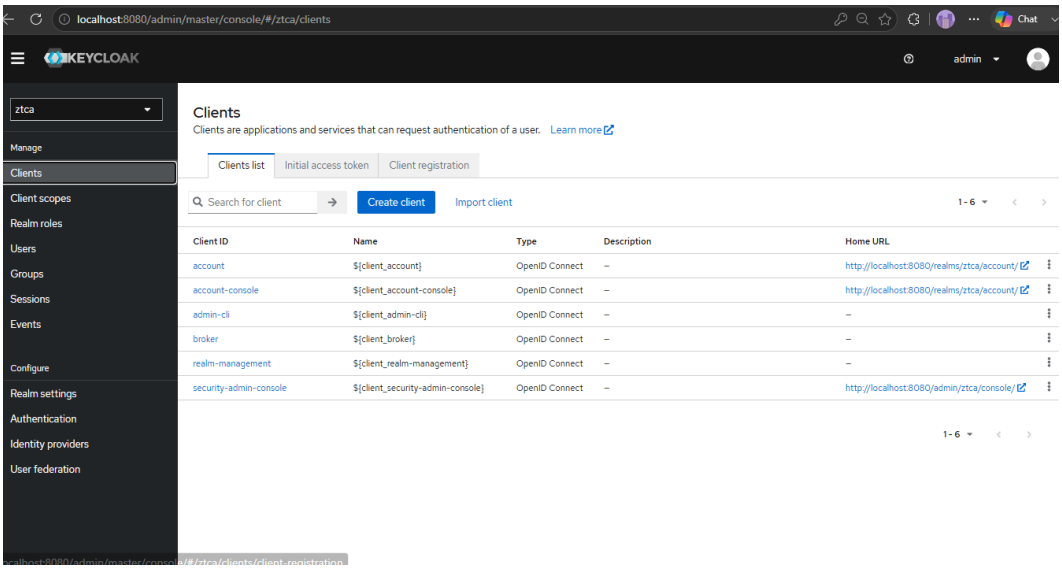


Fig 5.1 Click on create client in the Clients page

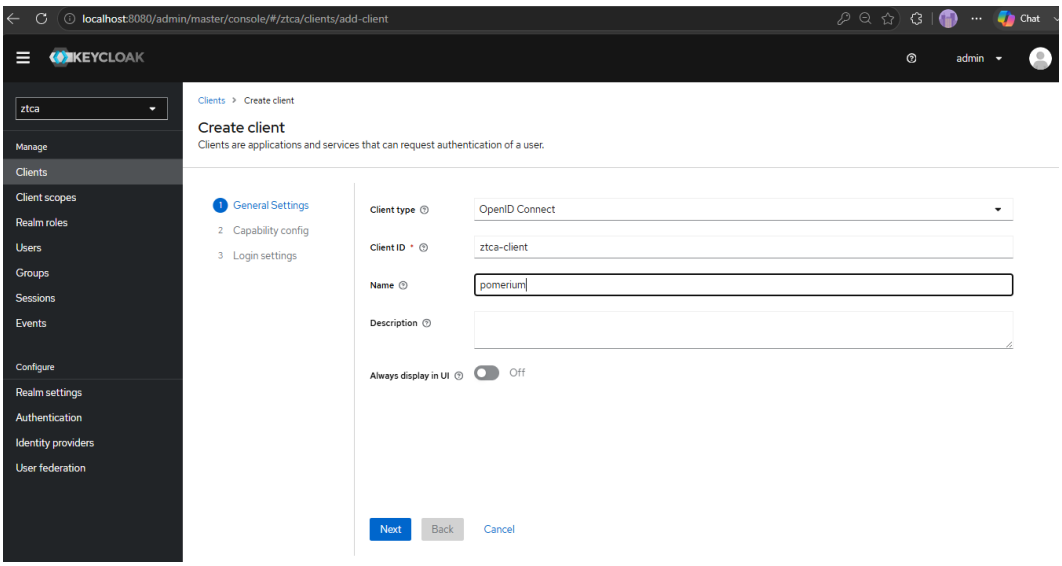


Fig 5.2 Naming and giving a client ID to the client

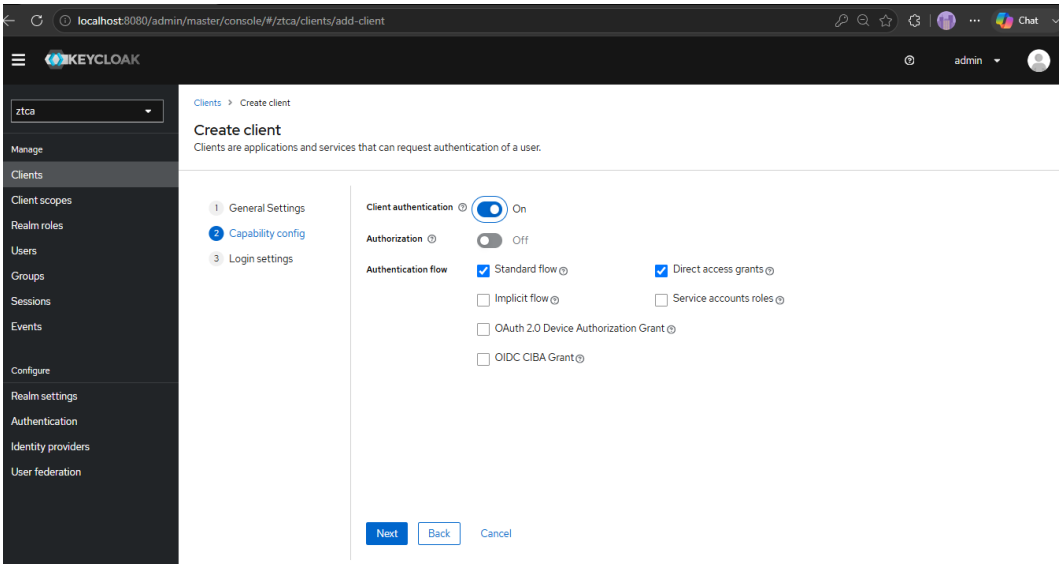


Fig 5.3 Configuring the capability settings

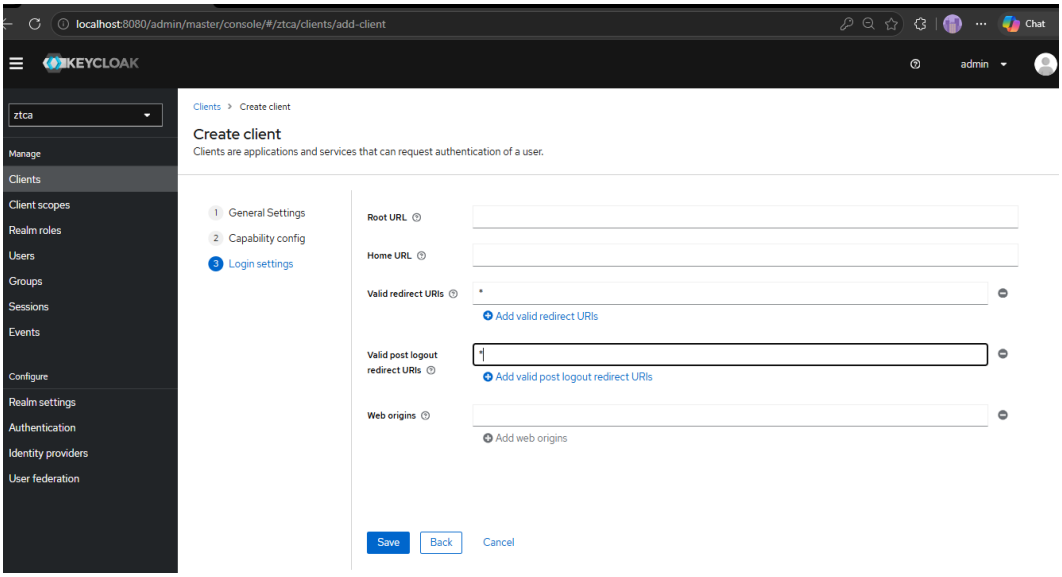


*Fig 5.4 Setting the Valid Redirect URI to * (wildcard) in the client settings.*

Client Settings:

Parameter	Value
Client ID	ztca-client
Client Protocol	openid-connect
Client Authentication	ON (confidential)
Standard Flow	Enabled
Direct Access Grants	Enabled

Valid Redirect URIs	* (wildcard)

2.5 Configuring the Client Secret

After creating the client, a Client Secret is auto-generated by Keycloak. This secret must be copied and pasted into the pomerium.yaml configuration file (idp_client_secret field) so that Pomerium can authenticate itself with Keycloak during OIDC token exchanges.

1. Inside the ztca-client settings, click the "Credentials" tab
2. Copy the generated Client Secret
3. Open docker/pomerium/config/pomerium.yaml
4. Paste the secret as the value of idp_client_secret

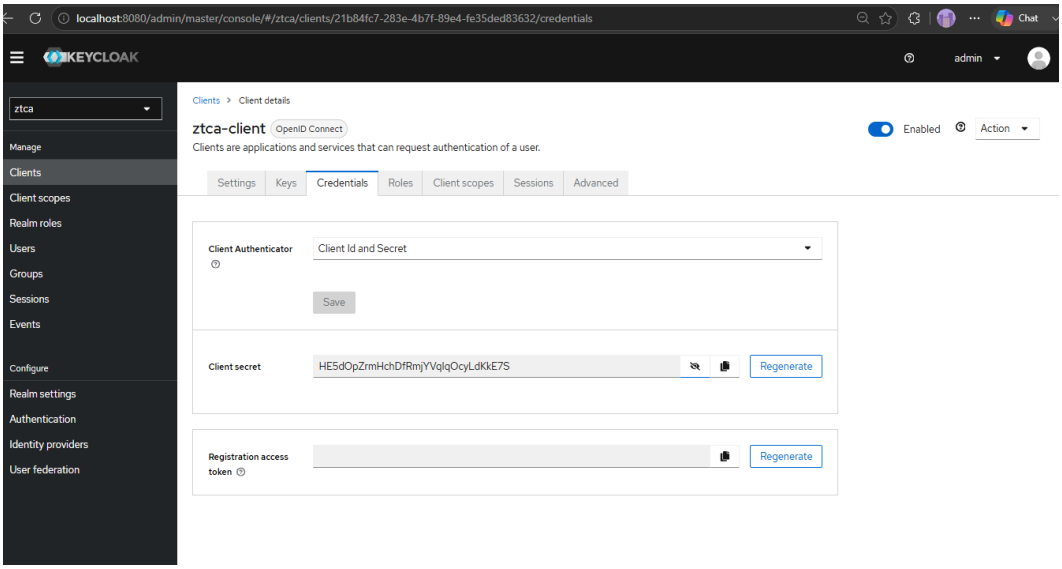


Fig 6.1 Client Credentials tab showing the generated Client Secret.

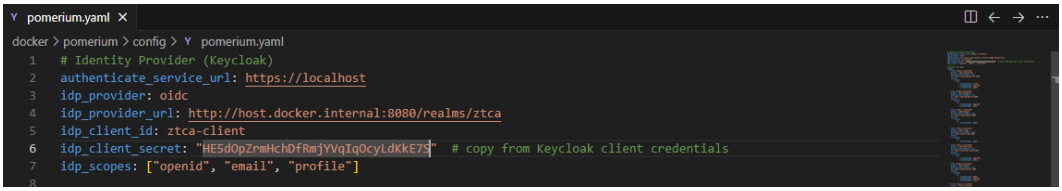


Fig 6.2 The Client Secret pasted into the pomerium.yaml configuration file.

2.6 Mapping Groups to the JWT Token

By default, Keycloak does not include a user's groups in the OIDC token. Pomerium needs the group claims to evaluate access policies, so a Protocol Mapper must be added to include group membership in the token.

Protocol Mapper Setup:

1. Inside the ztca-client settings, click the "Client scopes" tab
2. Click on the scope named "ztca-client-dedicated"
3. Click "Configure a new mapper"
4. Choose "Group Membership" from the list
5. Set Name to "groups" and Token Claim Name to "groups"
6. Toggle "Full group path" to OFF (so the claim is "admin" instead of "/admin")
7. Toggle "Add to ID token" and "Add to access token" to ON
8. Click "Save"

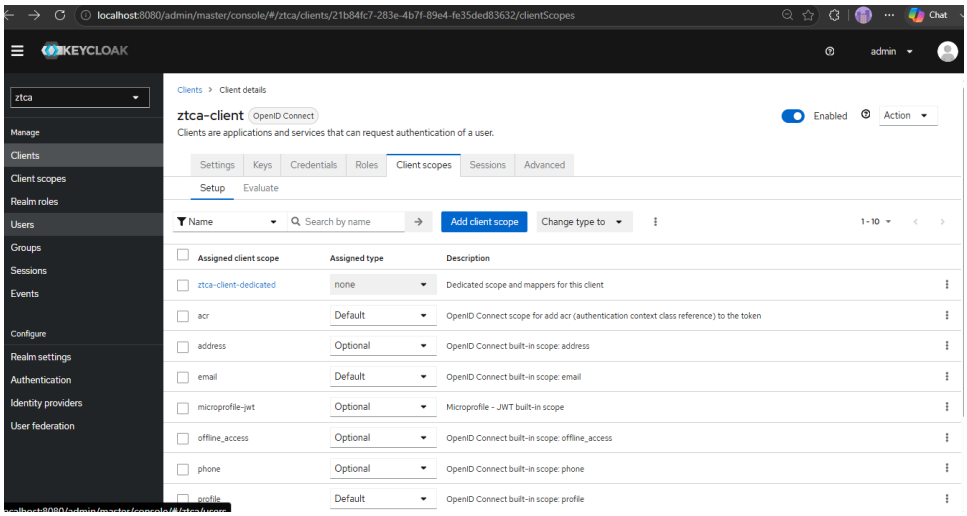


Fig 7.1 Selecting ztca-client-dedicated in the Client scopes

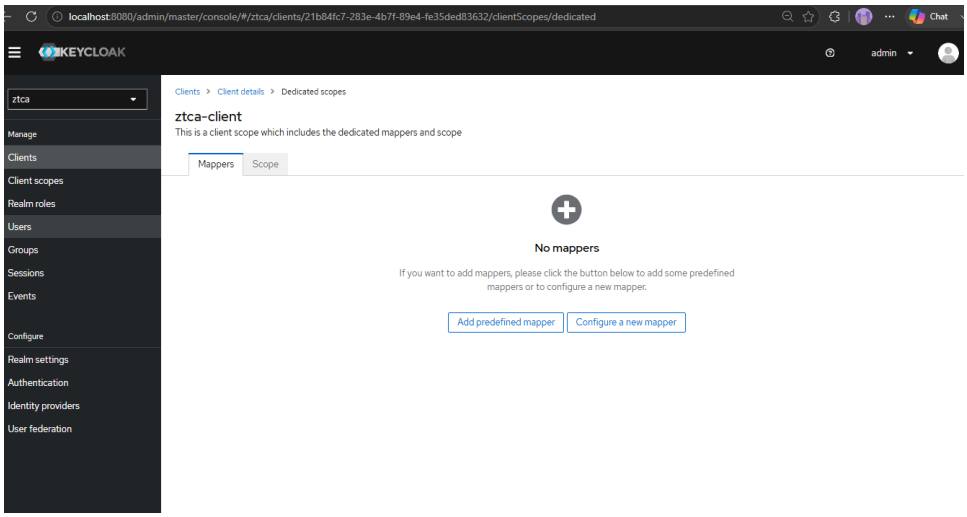


Fig 7.2 Selecting configure a new mapper

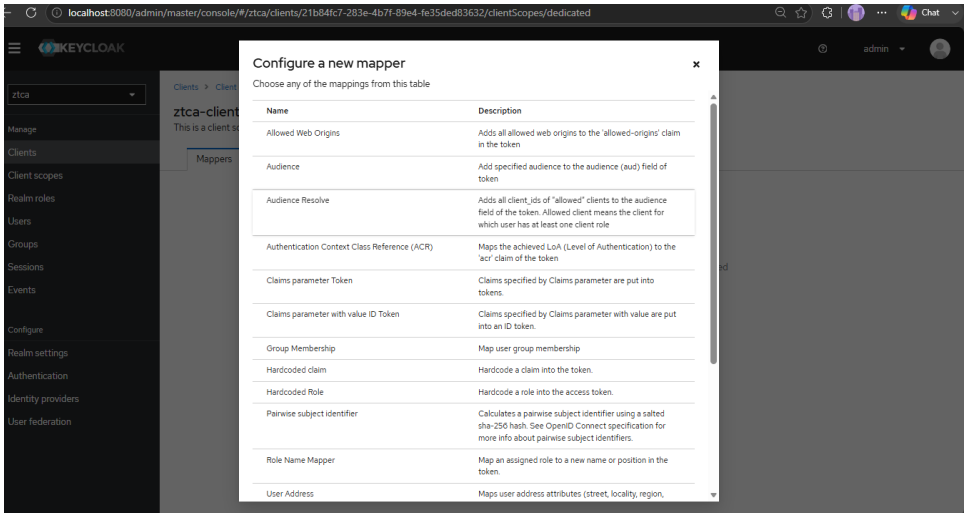


Fig 7.3 Select Group Membership

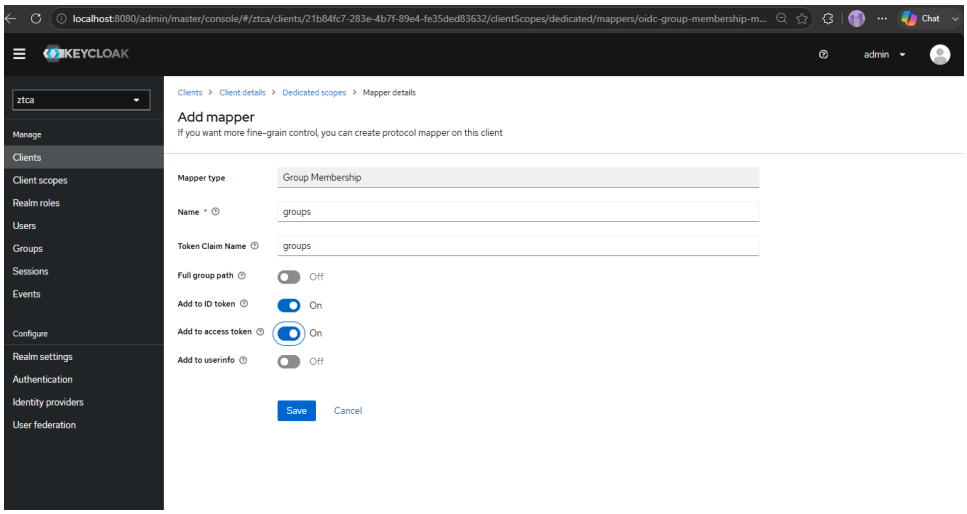


Fig 7.4 Naming and giving Token Client Name to the mapper

Milestone 3 – Pomerium Access Proxy

This milestone covers the configuration of Pomerium as the identity-aware reverse proxy in the Zero Trust Access Control system. Pomerium sits between the user and all protected applications, intercepting every request to authenticate users via Keycloak, evaluate group-based access policies, and forward only authorized traffic to the upstream services.

3.1 What is Pomerium?

Pomerium is an open-source, identity-aware access proxy that enables secure access to internal applications without a traditional VPN. It implements the Zero Trust principle by requiring every request to be authenticated and authorized before reaching the upstream service.

Unlike traditional reverse proxies that only handle traffic routing, Pomerium integrates directly with identity providers (such as Keycloak) via OpenID Connect and evaluates access policies on every request. It acts as the single entry point to all internal applications — all traffic on https://localhost passes through Pomerium.

Why Pomerium is Used in This Project:

Identity-Aware: Authenticates users through Keycloak OIDC before allowing access

Group-Based Policies: Uses Keycloak group claims (admin, engineer, intern) to decide access

No VPN Required: Users access internal services through their browser via https://localhost

Centralized Enforcement: All access decisions are made at a single proxy layer

Audit Logging: Every access attempt (allowed or denied) is logged for compliance

SSO Integration: Users authenticate once and access all authorized applications seamlessly

3.2 Configuration File (pomerium.yaml)

Pomerium is configured through a single YAML file located at docker/pomerium/config/pomerium.yaml. This file defines the identity provider connection, route mappings, and access policies. It is mounted into the Pomerium Docker container at /pomerium/config.yaml.

Identity Provider Settings:

The top section of pomerium.yaml connects Pomerium to Keycloak as its OIDC identity provider:

Setting	Value	Purpose
authenticate_service_url	https://localhost	Pomerium authentication endpoint
idp_provider	oidc	Use generic OIDC provider (Keycloak)
idp_provider_url	http://host.docker.internal:8080/realms/ztca	Keycloak OIDC discovery URL for the ztca realm
idp_client_id	ztca-client	Matches the Keycloak client registration
idp_client_secret	(from Keycloak Credentials tab)	Client secret for OIDC token exchange
idp_scopes	["openid", "email", "profile"]	OIDC scopes requested from Keycloak
log_level	debug	Verbose logging for development

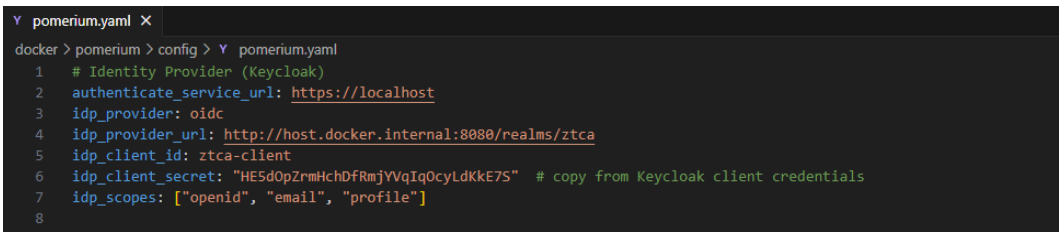


Fig 8 Snippet of pomerium.yaml showing the Identity Provider configuration section.

3.3 Route Definitions

Routes define the mapping between URL prefixes and internal upstream services. All routes use https://localhost as the base URL with prefix-based routing. Each route forwards traffic to an internal Docker service on port 3000. The preserve_host_header setting is enabled for all routes.

URL Path	Upstream Service	Authorized Groups
/main-portal/	http://main-portal:3000	admin, engineer, intern
/learning-portal/	http://learning-portal:3000	admin, engineer, intern
/dev-dashboard/	http://dev-dashboard:3000	admin, engineer
/internal-tools/	http://internal-tools:3000	admin, engineer
/admin-panel/	http://admin-panel:3000	admin only
/server-console/	http://server-console:3000	admin only
/audit-logs/	http://audit-logs:3000	admin only

All seven applications are isolated inside the private Docker network (ztca-net). None of them expose ports externally — the only way to reach them is through Pomerium on port 443. This ensures that even if an attacker gains network access, they cannot bypass the proxy.

3.4 Access Policies

Each route has a policy block that specifies which Keycloak groups are authorized to access it. Pomerium checks the "groups" claim in the user's JWT token (configured via the Protocol Mapper in Milestone 2) against the policy. If the user's group is listed in the policy's "or" block, access is granted; otherwise, Pomerium returns a 403 Forbidden response.

Policy Structure (from pomerium.yaml):

Each policy uses a claim/groups matcher with an "or" operator. For example, the Dev Dashboard policy allows access if the user belongs to either the "engineer" or "admin" group:

policy: allow → or → claim/groups: engineer, claim/groups: admin

This evaluates to: "allow if user is in engineer group OR admin group"

Complete Access Matrix:

Group	Main Portal	Learning	Dev	Tools	Admin	Server	Audit
admin	✓	✓	✓	✓	✓	✓	✓
engineer	✓	✓	✓	✓	✗ 403	✗ 403	✗ 403
intern	✓	✓	✗ 403	✗ 403	✗ 403	✗ 403	✗ 403

```yaml
routes:
  - from: https://localhost
    prefix: /learning-portal
    preserve_host_header: true
    to: http://learning-portal:3000
    policy:
      - allow:
          or:
            - claim/groups: intern
            - claim/groups: engineer
            - claim/groups: admin

  - from: https://localhost
    prefix: /dev-dashboard
    preserve_host_header: true
    to: http://dev-dashboard:3000
    policy:
      - allow:
          or:
            - claim/groups: engineer
            - claim/groups: admin

  - from: https://localhost
    prefix: /admin-panel
    preserve_host_header: true
    to: http://admin-panel:3000
    policy:
      - allow:
          or:
            - claim/groups: admin


  - from: https://localhost
    prefix: /audit-logs
    preserve_host_header: true
    to: http://audit-logs:3000
```

```
  policy:
    - allow:
        or:
          - claim/groups: admin

- from: https://localhost
  prefix: /internal-tools
  preserve_host_header: true
  to: http://internal-tools:3000
  policy:
    - allow:
        or:
          - claim/groups: engineer
          - claim/groups: admin

- from: https://localhost
  prefix: /main-portal
  preserve_host_header: true
  to: http://main-portal:3000
  policy:
    - allow:
        or:
          - claim/groups: admin
          - claim/groups: engineer
          - claim/groups: intern

- from: https://localhost
  prefix: /server-console
  preserve_host_header: true
  to: http://server-console:3000
  policy:
    - allow:
        or:
          - claim/groups: admin
```

3.5 Access Denied Response

When a user attempts to access a route for which their group is not listed in the policy, Pomerium blocks the connection at the network edge with a 403 Forbidden error. The user sees a Pomerium error page indicating that they are authenticated but not authorized.

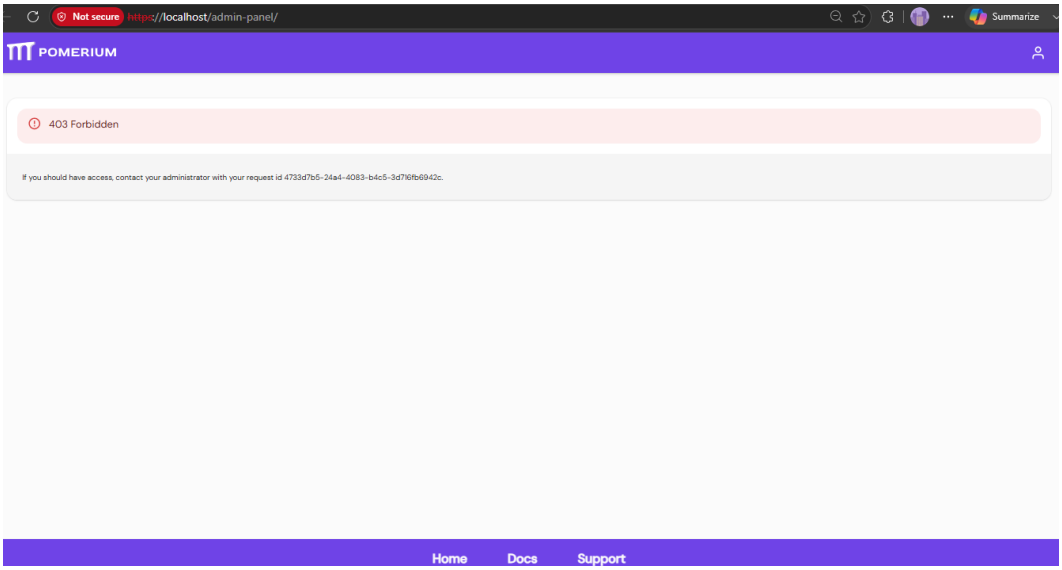


Fig 9 Pomerium 403 Forbidden page – shown when an unauthorized user attempts to access a restricted route.

3.6 Pomerium Session Dashboard

Pomerium provides a built-in session dashboard at https://localhost/.pomerium/ that displays the currently authenticated user's session details, including their JWT claims, group membership, and session expiration. Users can also sign out from this page to test with a different account.

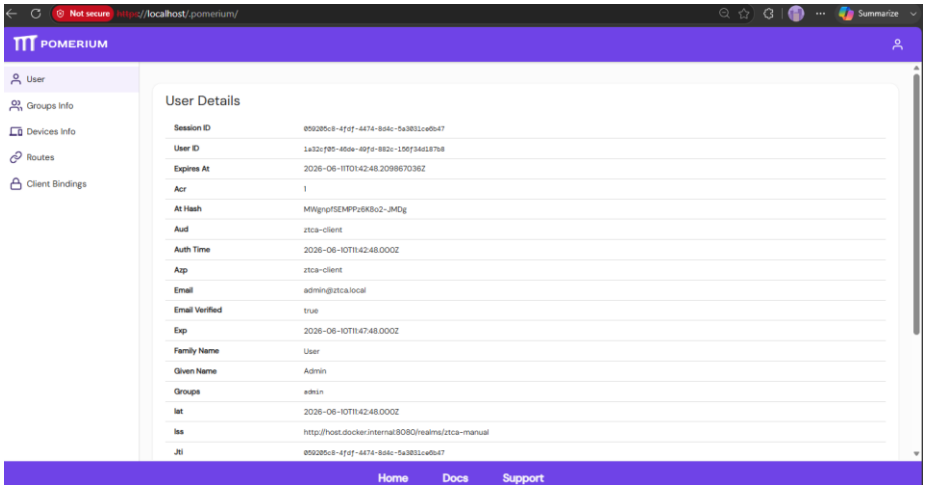


Fig 10.1 Pomerium session dashboard (https://localhost/.pomerium/) showing user details, group claims, and sign-out option.

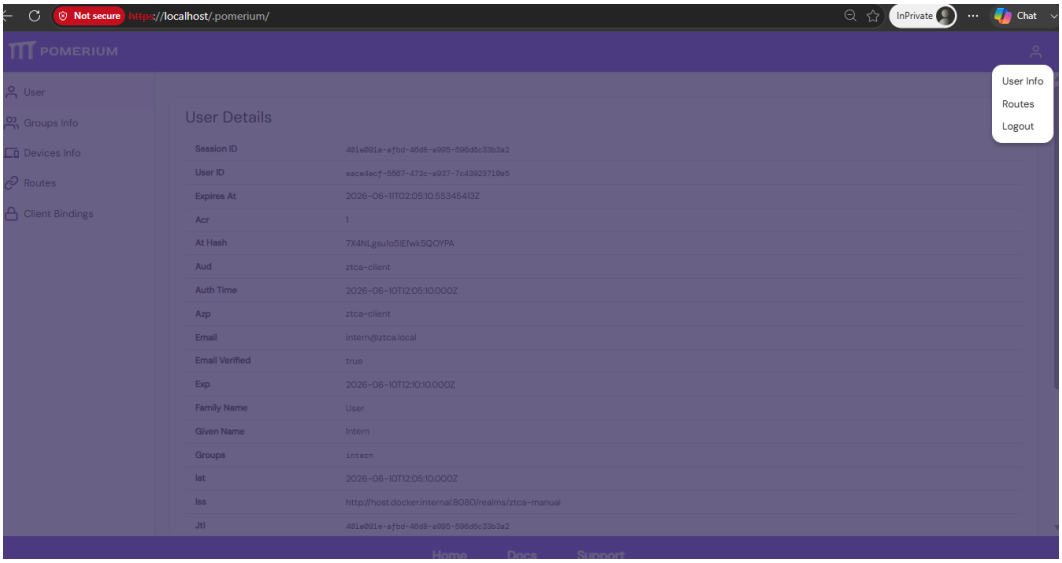


Fig 10.2 Pomerium session page showing the Sign Out option.

3.7 Authentication Flow

The complete authentication flow when a user accesses a protected application:

1. User navigates to https://localhost/<app-path>/ in the browser
2. Pomerium intercepts the request and checks for a valid session cookie
3. If no session exists, Pomerium redirects to Keycloak login page (ztca realm)
4. User enters credentials (e.g., engineer / engineer)
5. Keycloak validates credentials and issues an OIDC token with group claims
6. Pomerium receives the token, validates the signature, and creates a session cookie
7. Pomerium evaluates the route policy: checks if the user's groups match the allowed groups
8. If authorized, the request is forwarded to the upstream service; if not, 403 Forbidden

Milestone 4 – Frontend UI & Portal Design

This milestone covers the frontend user interface of the Zero Trust Access Control system. The project includes a main portal that serves as the central app launcher, plus six additional React applications — each simulating a real enterprise tool with working interactive functionality. All apps are built with React 18, Vite, and Lucide Icons, containerized with Docker, and served behind Pomerium. No authentication is built inside any app — all access control is handled externally by Pomerium and Keycloak.

4.1 Main Portal

The Main Portal (https://localhost/main-portal/) is the central launcher for all protected applications. After authenticating through Keycloak, users see a dashboard with cards linking to each application. Each card displays the app name, a brief description, the required access level, and a risk rating badge.

Portal Features:

Hero section with "Your Zero Trust Internal Access Portal" branding

Six application cards with access level badges and risk indicators

Search/filter functionality to find apps by name or access level

Zero Trust Access Flow diagram showing: User → Pomerium → Keycloak → Policy Check → App

Role Access Matrix table showing which groups can access which apps

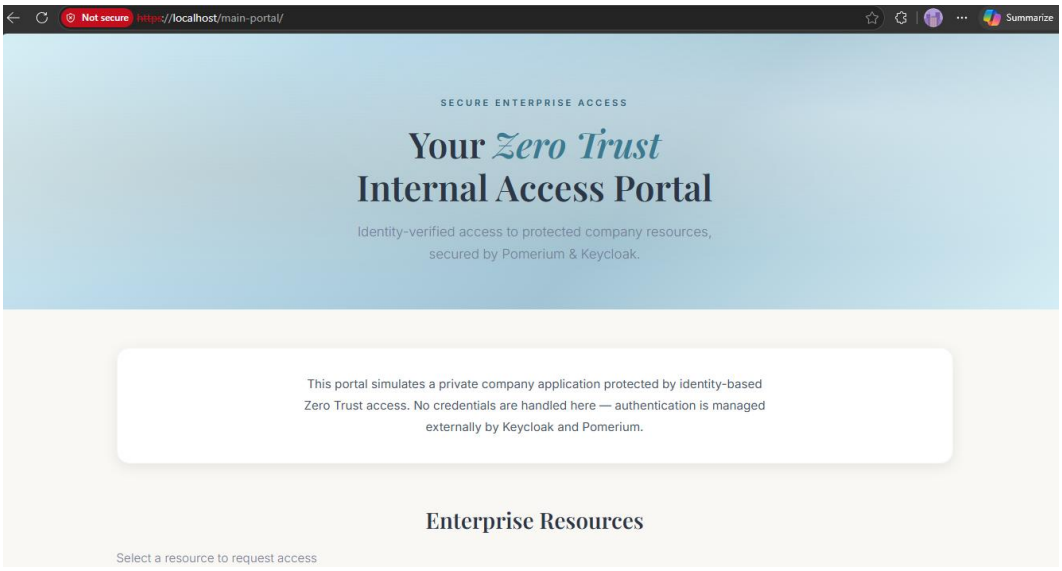


Fig 11 Main Portal page (https://localhost/main-portal/)

4.2 The Seven Applications

Seven standalone React applications simulate real enterprise tools. Each has working interactive features and a distinct visual identity. All are served on port 3000 inside the Docker network and are only reachable through Pomerium.

Application	URL Path	Access	Description & Interactive Features
Main Portal	/main-portal/	All users	Central app launcher. Search/filter apps, role matrix table, Zero Trust flow diagram.
Admin Panel	/admin-panel/	Admin only	Identity & access management. User table with search, group change dropdown, policy summary.
Server Console	/server-console/	Admin only	Server operations. Server status cards, restart simulation, active sessions, command console.
Audit Logs	/audit-logs/	Admin only	Security audit trail. Event table with filters (decision/group), search, CSV export, stats cards.
Dev Dashboard	/dev-dashboard/	Admin, Engineer	CI/CD & engineering. Pipeline trigger (queued→running→success), API health, ticket filter, deployment timeline.
Internal Tools	/internal-tools/	Admin, Engineer	Utilities & diagnostics. API tester form, tool buttons with output, service registry, environment selector.
Learning Portal	/learning-portal/	All users	Zero Trust training. Module progress checklist, 5-question quiz with scoring, glossary cards.

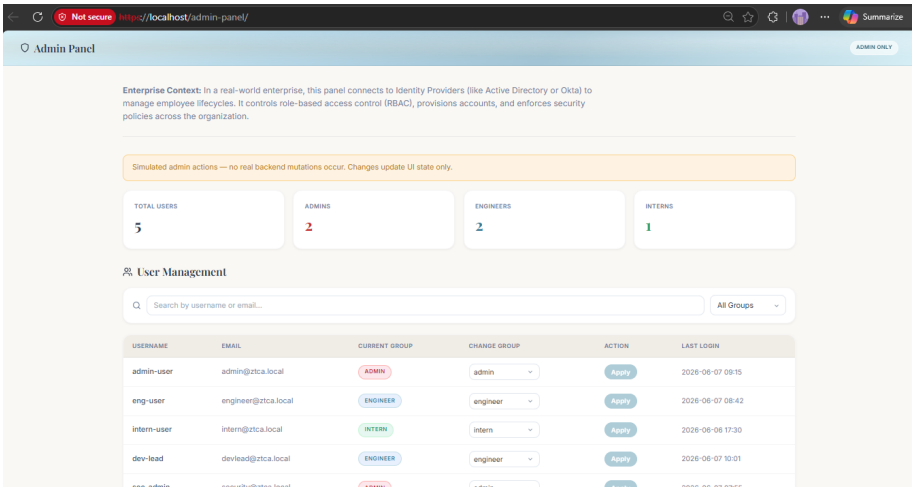


Fig 12.1 Admin Portal

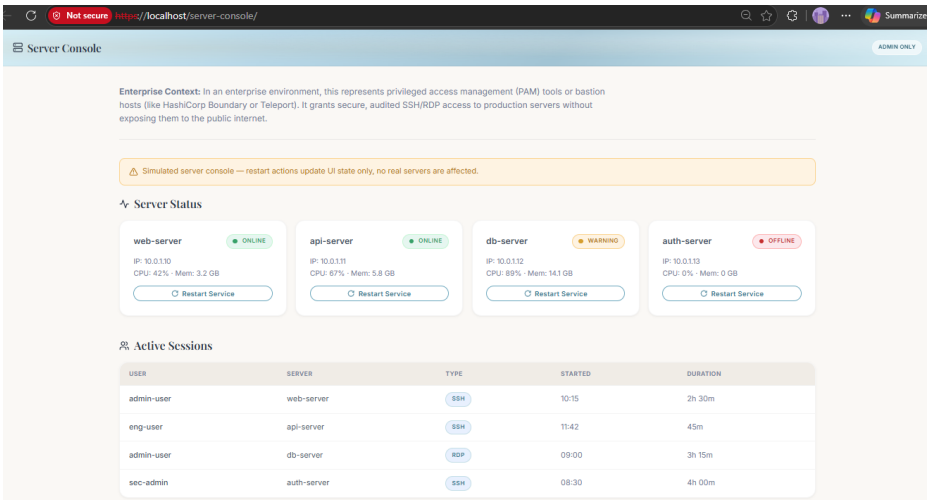


Fig 12.2 Server Console

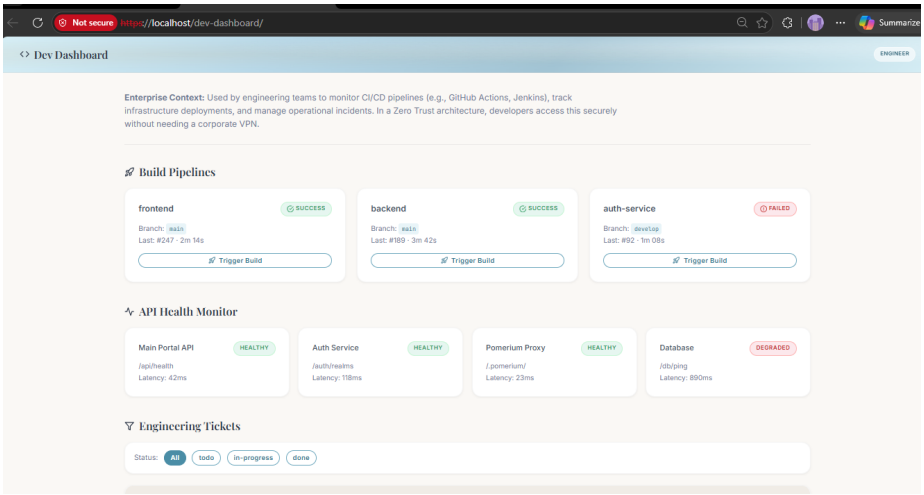


Fig 12.3 Dev Dashboard

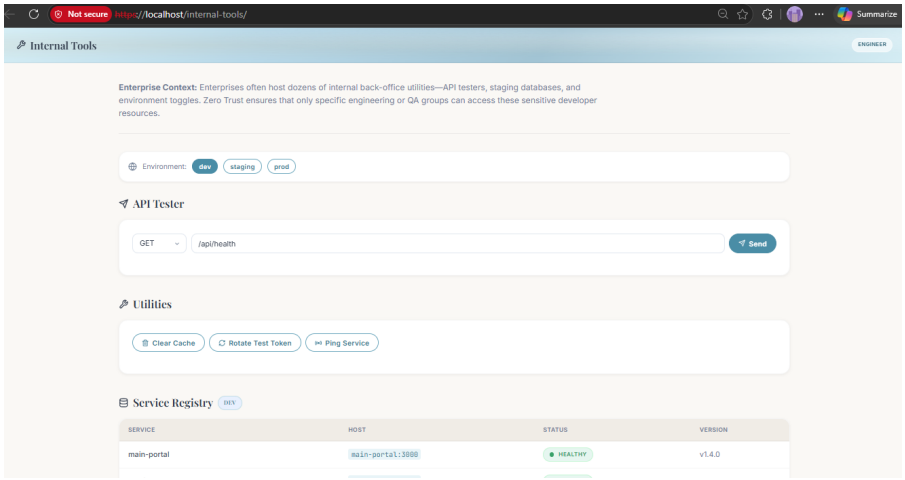


Fig 12.4 Internal Tools

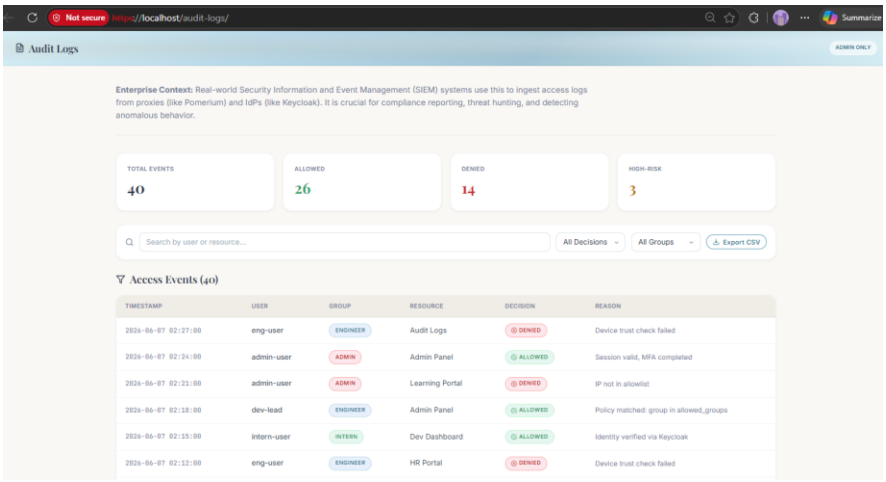


Fig 12.5 Audit Logs

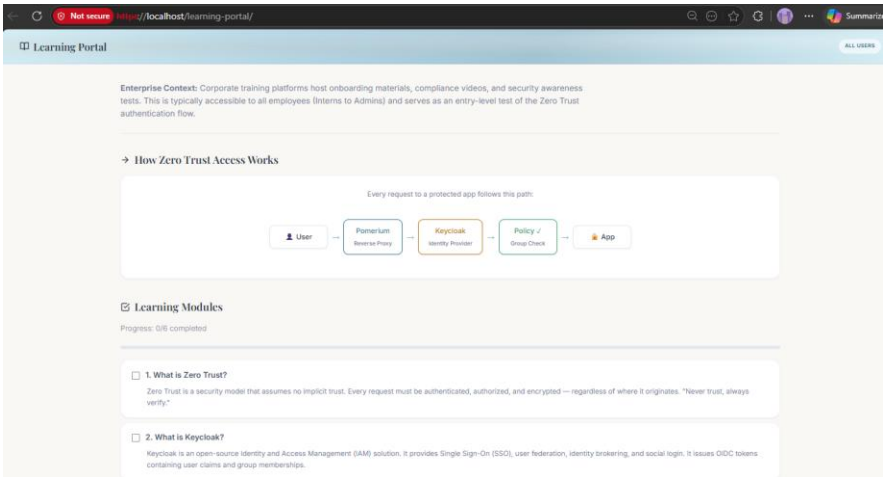


Fig 12.6 Learning Portal

4.3 Keycloak Authentication Page

When a user navigates to any application URL (e.g., https://localhost/main-portal/) without an active session, Pomerium automatically redirects them to the Keycloak login page for the "ztca" realm. The user enters their credentials (e.g., engineer / engineer) and Keycloak authenticates them and redirects back to the requested application.

Login Flow:

1. User opens https://localhost/main-portal/ in the browser
2. Pomerium detects no active session and redirects to Keycloak
3. Keycloak displays the login page for the "ztca" realm
4. User enters username and password
5. On success, Keycloak redirects back to Pomerium with an authorization code
6. Pomerium creates a session cookie and displays the requested application

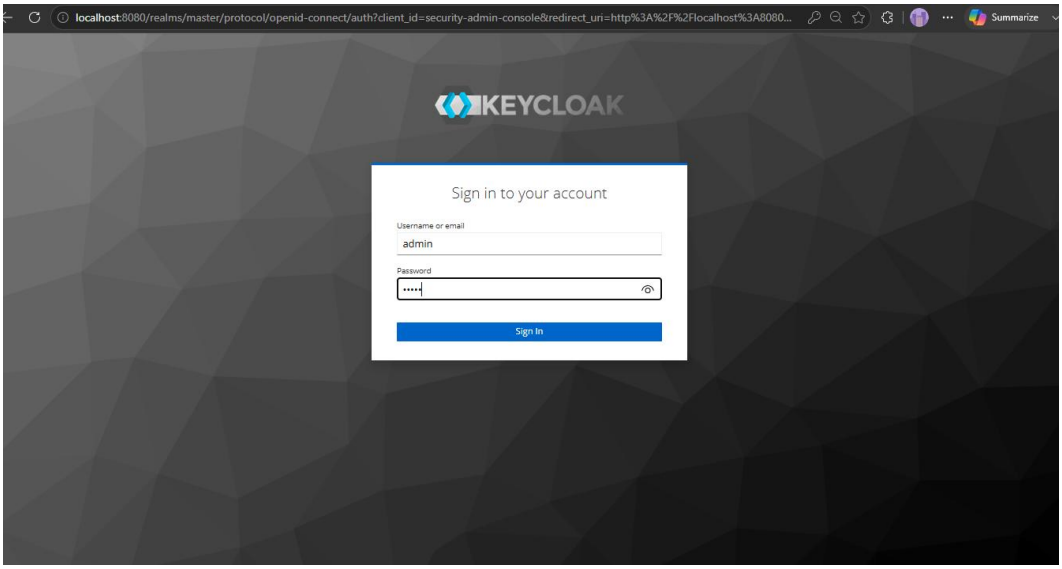


Fig 13 Keycloak login page

4.4 Role Access Matrix

The Main Portal includes a built-in Role Access Matrix that shows which Keycloak group can access which application. This matrix is enforced by Pomerium's policies:

Group	Portal	Admin	Server	Audit	Dev	Tools	Learning
Admin	✓	✓	✓	✓	✓	✓	✓
Engineer	✓	✗	✗	✗	✓	✓	✓
Intern	✓	✗	✗	✗	✗	✗	✓

4.5 Accessing the Applications

All access goes through Pomerium via https://localhost. A trailing slash must be included at the end of each URL to prevent redirect issues:

Main Portal: https://localhost/main-portal/

Admin Panel: https://localhost/admin-panel/

Server Console: https://localhost/server-console/

Audit Logs: https://localhost/audit-logs/

Dev Dashboard: https://localhost/dev-dashboard/

Internal Tools: https://localhost/internal-tools/

Learning Portal: https://localhost/learning-portal/

Pomerium Session: https://localhost/.pomerium/ (view JWT claims, sign out)

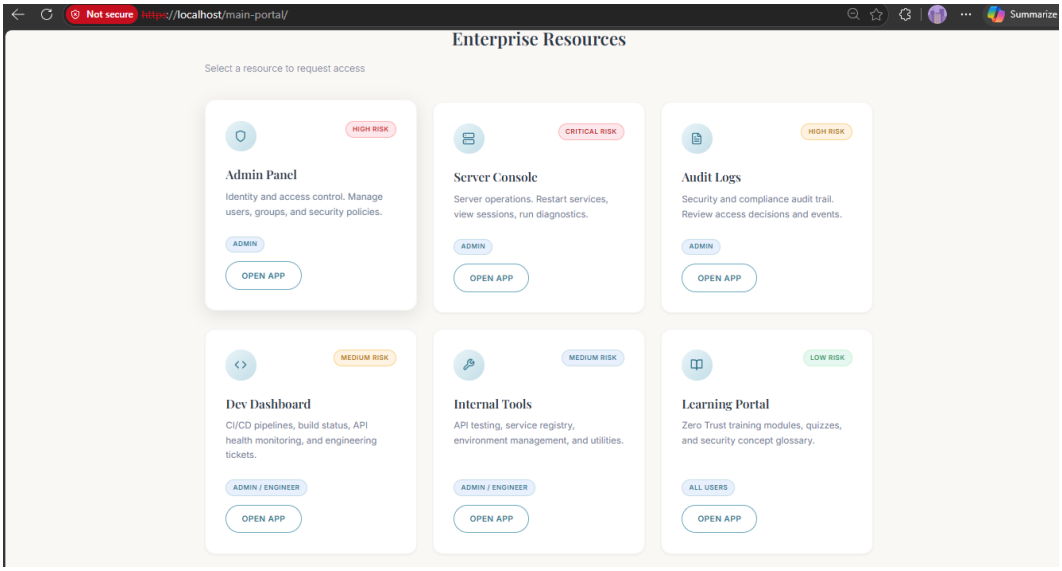


Fig 14 Browser showing access to one of the applications through https://localhost.

Milestone 5 – Testing & Development

5.1 Test Data Preparation

Three test user accounts are pre-seeded in Keycloak via realm-export.json. Each belongs to a different group, representing a distinct access tier in the Zero Trust system:

Username	Password	Group	Expected Access
admin	admin	admin	All 7 applications
engineer	engineer	engineer	Main Portal, Dev Dashboard, Internal Tools, Learning Portal
intern	intern	intern	Main Portal and Learning Portal only

5.2 Test Execution Steps

5.2.1 Prerequisites

Ensure the Windows hosts file (C:\Windows\System32\drivers\etc\hosts) contains the entry:

127.0.0.1 host.docker.internal

5.2.2 Launch the Infrastructure

Start the Keycloak IAM server and Pomerium proxy:

cd docker

docker-compose up -d

Keycloak automatically imports users and roles from realm-export.json on first boot. Wait 30–60 seconds for Keycloak to fully initialize.

5.2.3 Launch the Applications

Start the seven React applications:

cd ztca-portal-clean

docker-compose up -d --build

5.2.4 Test Workflow

1. Navigate to https://localhost/main-portal/ — verify redirect to Keycloak login.

2. Log in as admin (admin / admin) — verify Main Portal loads with all 6 app cards.

3. Click "Admin Panel" — verify access granted (admin group).

4. Click "Server Console" — verify access granted (admin group).

5. Click "Audit Logs" — verify access granted (admin group).

6. Click "Dev Dashboard" — verify access granted.

7. Click "Internal Tools" — verify access granted.

8. Click "Learning Portal" — verify access granted.

9. Sign out at https://localhost/.pomerium/ and log in as engineer (engineer / engineer).

10. Navigate to Dev Dashboard — verify access granted.

11. Navigate to Admin Panel — verify 403 Forbidden response.

12. Navigate to Server Console — verify 403 Forbidden response.

13. Sign out and log in as intern (intern / intern).

14. Navigate to Learning Portal — verify access granted.

15. Navigate to Dev Dashboard — verify 403 Forbidden response.

16. Navigate to Admin Panel — verify 403 Forbidden response.

5.3 Functional Testing Results

The following scenarios were tested and validated:

Unauthenticated Access Redirect: Navigating to any application URL without a session correctly redirects to the Keycloak login page for the "ztca" realm.

SSO Login: After entering valid credentials, the user is redirected back to the requested application. Navigating to other apps does not require re-authentication (SSO active).

Admin Full Access: The admin user successfully accessed all seven applications without any 403 errors.

Engineer Partial Access: The engineer user accessed Main Portal, Dev Dashboard, Internal Tools, and Learning Portal. Attempts to access Admin Panel, Server Console, and Audit Logs returned 403 Forbidden.

Intern Minimal Access: The intern user accessed Main Portal and Learning Portal only. All other applications returned 403 Forbidden.

Session Sign-Out: Visiting https://localhost/.pomerium/ and clicking Sign Out correctly terminates the session. The next request redirects to the Keycloak login page.

Invalid Credentials: Entering incorrect username or password on the Keycloak login page displays an error message and no session is created.

Milestone 6 – Results & Evaluation

System Output:

The Zero Trust Access Control system successfully enforces identity-aware, group-based access control across all seven protected applications. Every request passes through Pomerium, which validates the user's session with Keycloak and checks their group claims against the YAML policy. The system correctly grants or denies access based on the user's Keycloak group membership.

Performance Evaluation:

The complete authentication flow (Keycloak login, token exchange, session creation) completes within 2–3 seconds on first access. Subsequent requests with an active session are handled in under 100ms as Pomerium validates the session cookie locally. The Docker stack (Keycloak + Pomerium + 7 apps + backend) starts within 30–60 seconds on a standard development machine.

Access Control Validation:

All test scenarios passed successfully. The access control matrix was validated end-to-end with all three test users across all seven applications. Pomerium correctly evaluated the Keycloak-issued JWT group claims against each route's policy. No unauthorized access was observed during testing.

Screenshots — Authentication Workflow

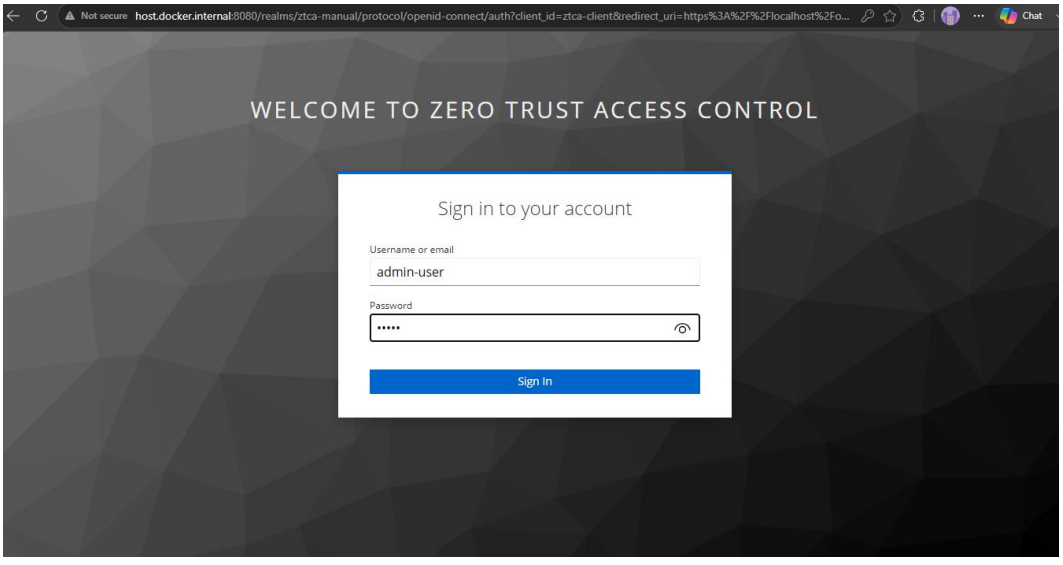


Fig 15 Keycloak login page displayed when navigating to https://localhost/main-portal/ without a session.

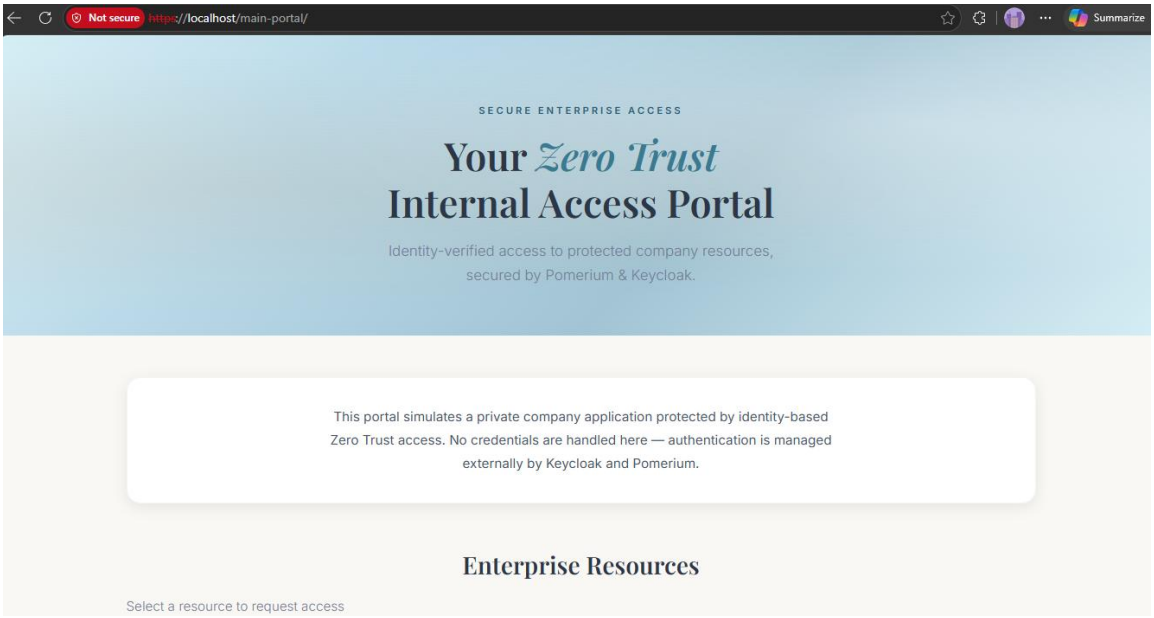


Fig 16 Successful authentication – Main Portal loads after logging in as admin.

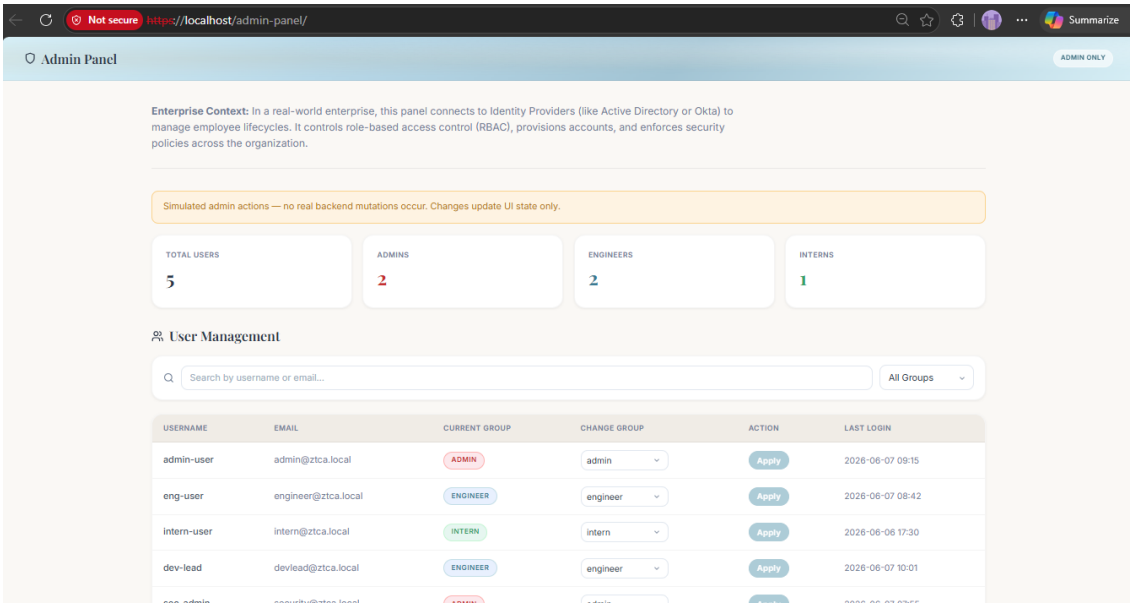


Fig 17 SSO in action – accessing Admin Panel without re-entering credentials.

Screenshots — Access Denied Responses

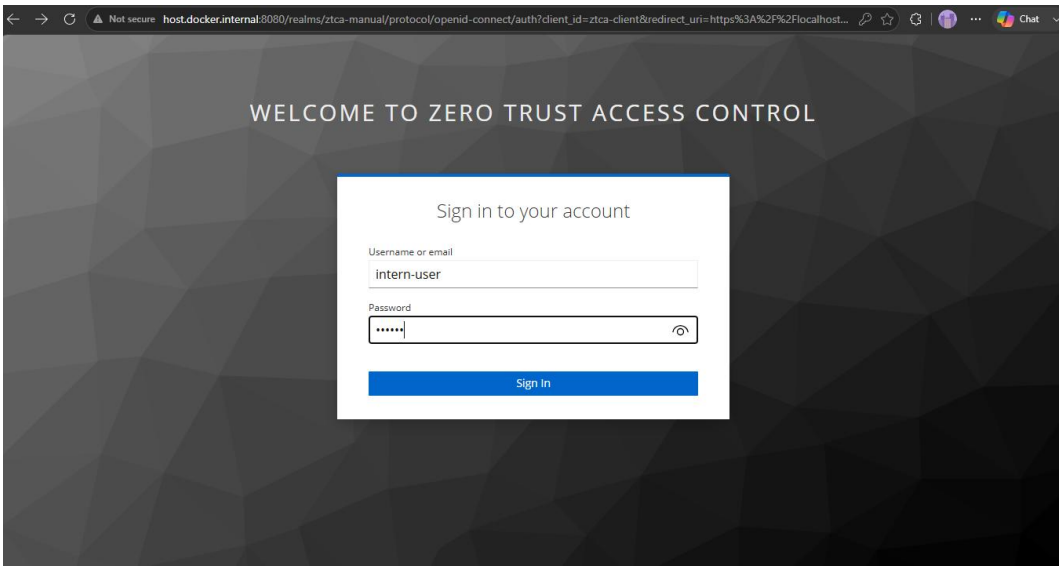


Fig 18 Intern Logging into Main Portal

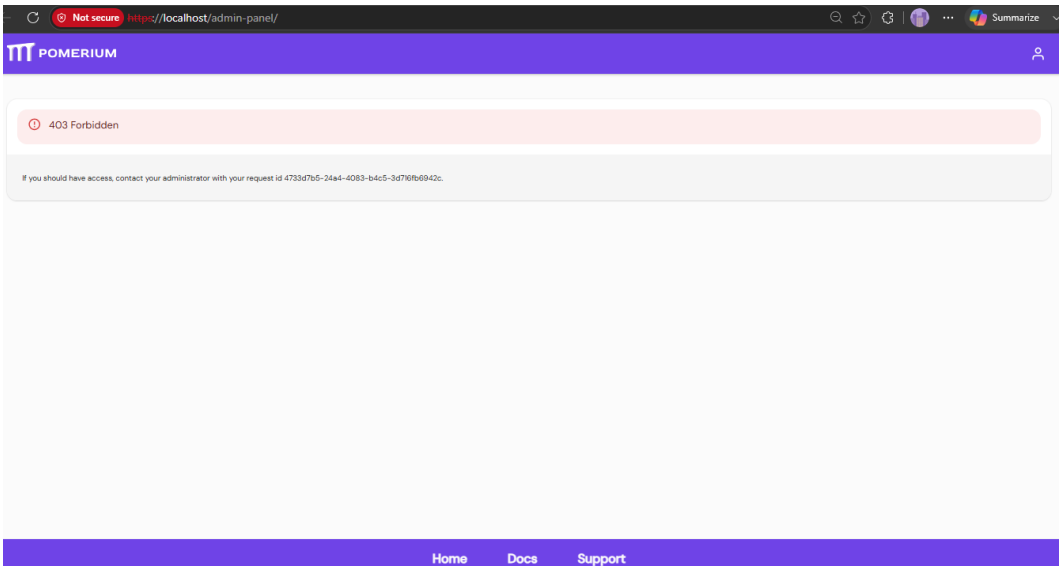


Fig 19 Intern user denied access to Admin Panel – 403 Forbidden page.

Screenshots — Pomerium Session Dashboard

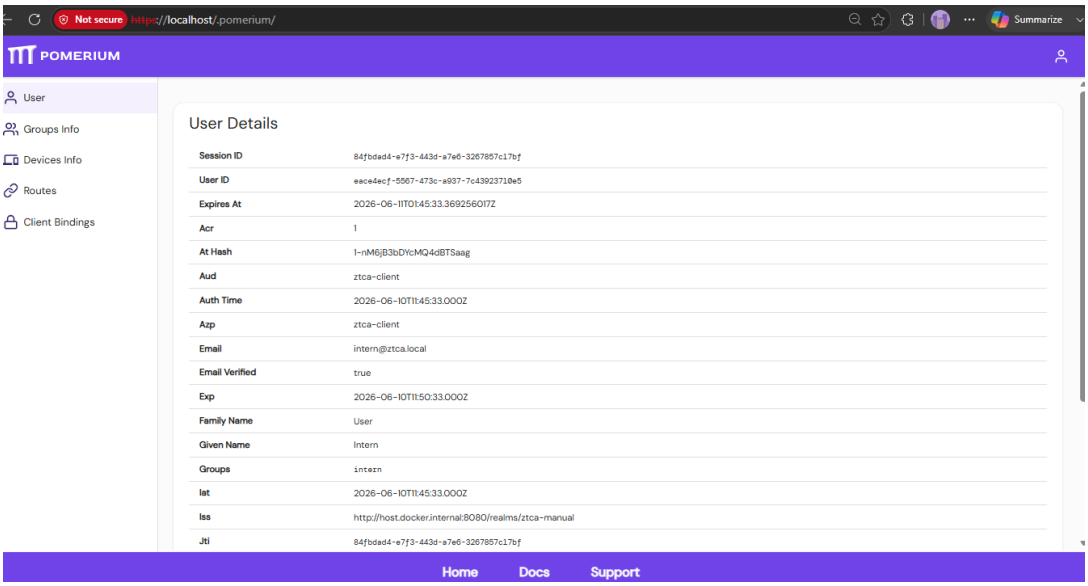


Fig 20 Pomerium session dashboard (https://localhost/.pomerium/) showing user JWT claims and group membership.

Screenshots — Docker Container Status

```
PS D:\Flipnow-Projects\Cybersecurity\ztca-project\docker> docker ps
CONTAINER ID   IMAGE                                COMMAND                  CREATED       STATUS       PORTS
                                    NAMES
45fa24187d9d   pomerium/pomerium:latest             "/bin/pomerium --con…"   31 hours ago   Up 2 hours   0.0.0.0:80->80/tcp, [::]:80->80/tcp,
0.0.0.0:443->443/tcp, [::]:443->443/tcp   pomerium
a6398c59f73d   quay.io/keycloak/keycloak:21.1.1     "/opt/keycloak/bin/k…"   31 hours ago   Up 2 hours   0.0.0.0:8080->8080/tcp, [::]:8080->80
80/tcp                              keycloak
a18aaea5abc1   ztca-portal-clean-admin-panel        "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-admin-panel-1
394e02d68957   ztca-portal-clean-learning-portal    "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-learning-portal-1
73e5a76d59c2   ztca-portal-clean-server-console     "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-server-console-1
ced023dbd8f2   ztca-portal-clean-main-portal        "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-main-portal-1
a88411ceffef   ztca-portal-clean-audit-logs         "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-audit-logs-1
97cf8add1599   ztca-portal-clean-dev-dashboard      "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-dev-dashboard-1
142bf813e8a6   ztca-portal-clean-internal-tools     "/docker-entrypoint.…"   33 hours ago   Up 7 hours   80/tcp, 3000/tcp
                                    ztca-portal-clean-internal-tools-1
6119d02734c1   ztca-portal-clean-backend            "docker-entrypoint.s…"   33 hours ago   Up 7 hours   0.0.0.0:5000->5000/tcp, [::]:5000->50
00/tcp                              ztca-portal-clean-backend-1
PS D:\Flipnow-Projects\Cybersecurity\ztca-project\docker>
```

Fig 21 Terminal output of docker-compose ps showing all containers running.

Screenshots — Applications

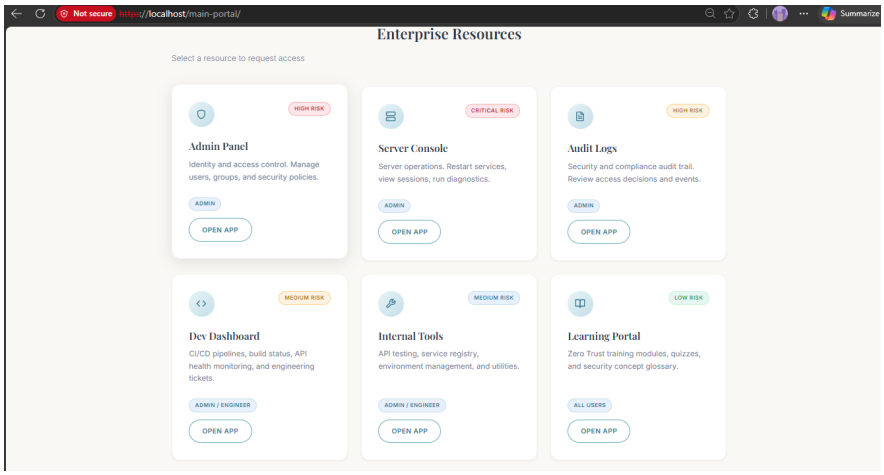


Fig 22 Main Portal page with app launcher cards and Zero Trust flow diagram.

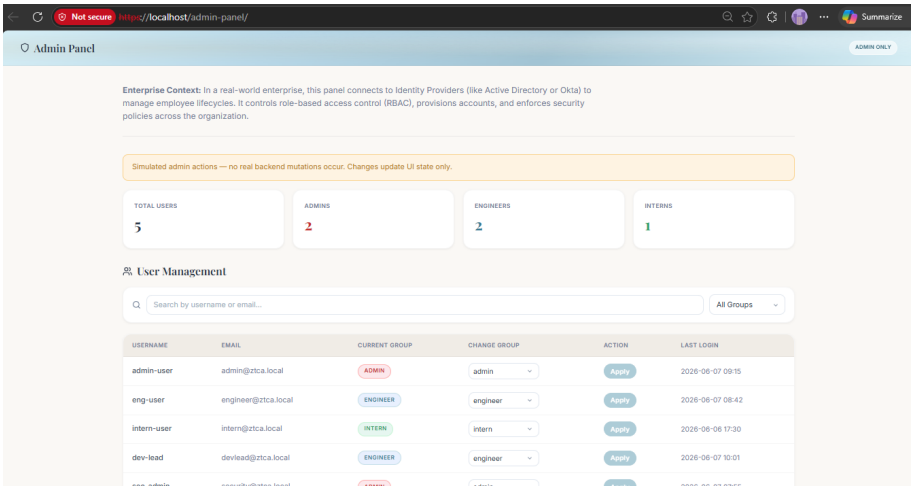


Fig 23 Admin Panel – user management dashboard with policy summary.

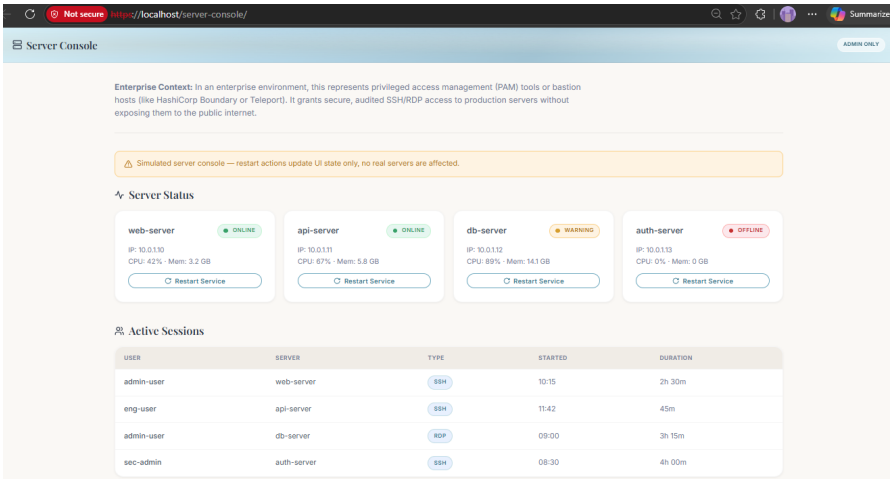


Fig 24 Server Console – server status cards and command console.

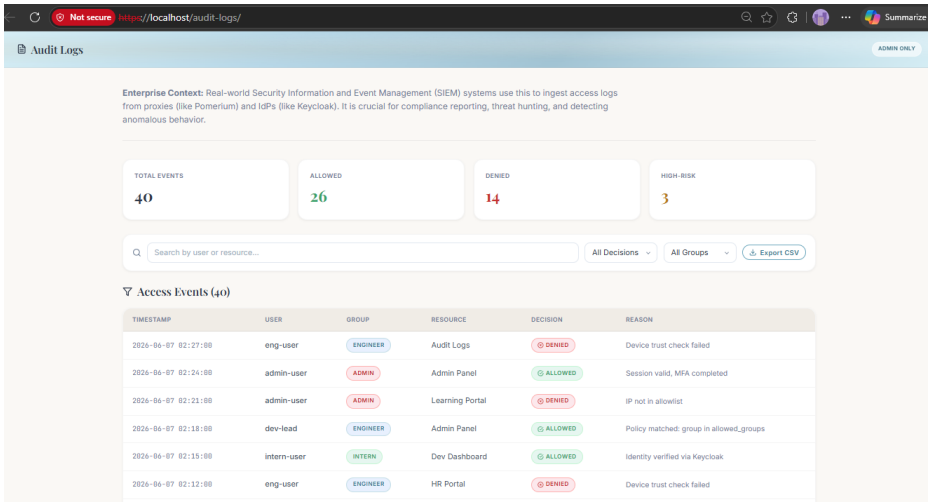


Fig 25 Audit Logs – event table with decision filters and CSV export.

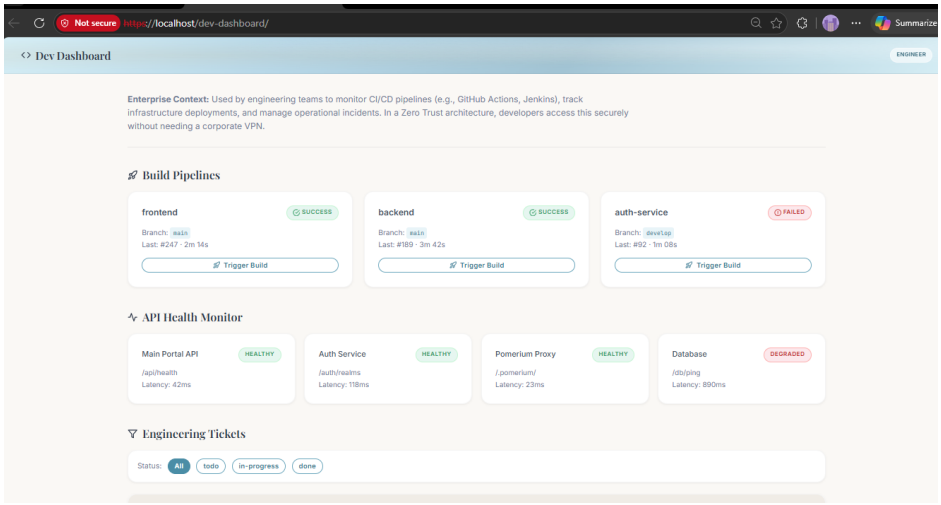


Fig 26 Dev Dashboard – CI/CD pipelines, API health, and engineering tickets.

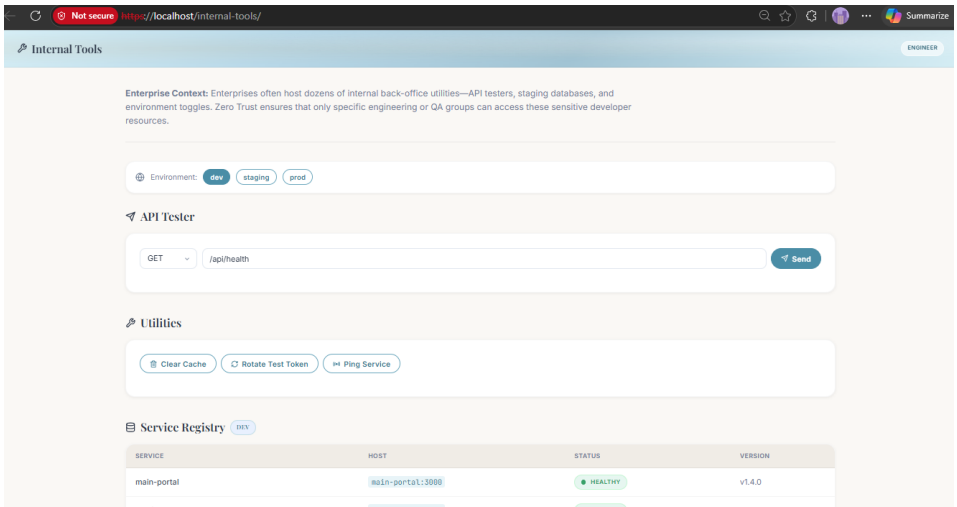


Fig 27 Internal Tools – API tester, service registry, and environment selector.

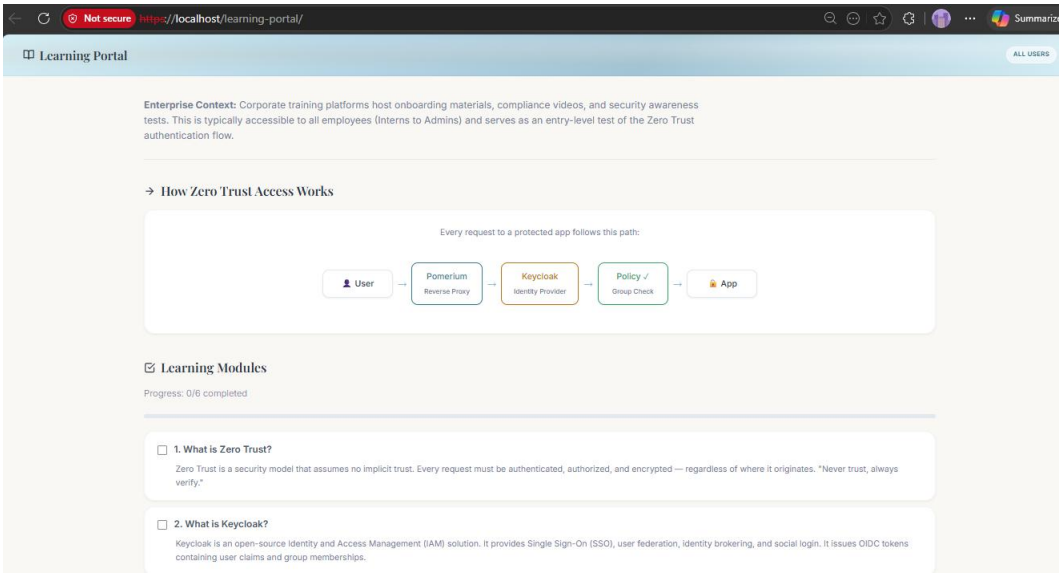


Fig 28 Learning Portal – Zero Trust training modules and quiz.

ADVANTAGES & LIMITATIONS

Advantages

Zero Trust Enforcement: Every request is authenticated and authorized at the proxy layer, eliminating implicit trust for internal network traffic.

Centralized Identity Management: Keycloak provides a single source of truth for user identities and group memberships across all seven applications.

Group-Based Policies: Pomerium's declarative YAML policies make access rules auditable, version-controllable, and easy to modify without touching application code.

No In-App Auth: Applications are completely unaware of authentication — they receive pre-authenticated traffic from Pomerium, simplifying application development.

SSO Experience: Users authenticate once and access all authorized applications seamlessly.

Container-Native: The entire stack runs as Docker containers with a shared network, enabling single-command deployment.

Complete Audit Trail: Pomerium logs every access decision with user identity, resource, and timestamp for compliance monitoring.

Limitations

Self-Signed TLS: The development environment uses Pomerium's auto-generated certificates, causing browser security warnings.

No MFA: Password-only authentication is used. Production would require multi-factor authentication.

Static Policies: Policy changes require editing pomerium.yaml and restarting the proxy.

hosts File Dependency: The Windows hosts file must be manually configured for host.docker.internal resolution.

FUTURE ENHANCEMENTS

Multi-Factor Authentication (MFA): Integrate Keycloak MFA with OTP or WebAuthn.

Cloudflare Tunnel: Add secure ingress without exposing ports publicly.

Tailscale: Add mesh networking for internal service communication.

Production TLS: Integrate with Let's Encrypt for automatic certificate management.

Centralized Logging: Integrate with ELK Stack or Grafana Loki for log aggregation.

Dynamic Policies: Build a policy management UI for real-time policy changes.

Device Trust: Add device posture checks as additional policy conditions.

CONCLUSION

Summary of Implementation:

The Zero Trust Access Control with Keycloak and Pomerium project has been successfully developed as a complete, containerized security infrastructure that demonstrates practical implementation of Zero Trust Architecture using open-source tools. The system enforces the principle of "never trust, always verify" by requiring authentication and policy-based authorization for every access request to protected services.

Technical Achievements:

A major achievement is the seamless integration between Keycloak (Identity Provider) and Pomerium (Access Proxy) through OpenID Connect. The group claims in the OIDC token issued by Keycloak are correctly evaluated by Pomerium's policy engine via the claim/groups matcher, enabling fine-grained group-based access control without any custom authentication code in the applications. The declarative YAML-based policy configuration makes access rules auditable, version-controllable, and easy to modify.

Key Achievements:

Successfully deployed Keycloak as a centralized Identity Provider with OIDC support, managing user identities, groups (admin, engineer, intern), and authentication flows

Configured Pomerium as an identity-aware reverse proxy with route-specific group-based policies across seven protected applications

Implemented a multi-tier access model where admin has full access, engineer has partial access, and intern has minimal access — enforcing the principle of least privilege

Achieved SSO functionality where users authenticate once through Keycloak and access all authorized applications without repeated login prompts

Built seven interactive React micro-frontend applications simulating real enterprise tools, all containerized with Docker and served behind Pomerium

Verified complete access control enforcement through comprehensive testing with all three user accounts across all seven applications

Established an audit trail through Pomerium access logs for compliance and security monitoring

Practical Impact:

The project validates that Zero Trust Architecture can be practically implemented using freely available, open-source tools, making it accessible to organizations of all sizes. The containerized deployment model ensures that the solution can be easily replicated and scaled in production environments. By replacing traditional VPN-based access with identity-aware proxy-based access, organizations can significantly improve their security posture while maintaining a seamless user experience through Single Sign-On.

APPENDIX

A.1 Docker Commands Reference

Command	Description
docker-compose up -d	Start all services in detached mode
docker-compose down	Stop and remove all containers
docker-compose logs -f	Follow live logs from all services
docker-compose ps	List running containers and their status
docker-compose restart pomerium	Restart Pomerium after config changes

A.2 Key Configuration Files

File	Purpose
docker-compose.yml	Service definitions, networks, and volumes
docker/pomerium/config/pomerium.yaml	Pomerium routes, policies, and OIDC settings

A.3 Source Code - GitHub Repository